第五周周末及第六周周报

- 2026/10/16
  - 回顾复习三叶草视频
    - 第一个视频：三叶草二进制招新培训2 - 简单的加密算法
    - 第二个视频：三叶草二进制招新培训3 - 简单的加密算法2
    - 今日学习总结（附录）
    - 明日目标
- 2026/10/17
  - 先做几道题
    - 简单XOR
    - BABBYRE
    - 第三题
    - 第四题
    - 第五题
    - 结语
- 2026/10/18（10/19进行补充）
  - base64
  - Base64变表逆向
  - RC4
  - TEA家族（10/22）
    - TEA
    - BlockTEA
    - XTEA
    - XXTEA
- 2026/10/19
  - IDA动态调试
    - IDA动态调试秒RC4
- 2026/10/22（10/20修复IDA插件问题，下坏了，重新下的byte老师的 10/21写作业，太多了，作文，英语...
  - IDA代码修复与数组识别(看不太懂)

代码修复：常见问题与对应的处理办法

必须记熟的快捷键

2026/10/23

[NISACTF 2022]string

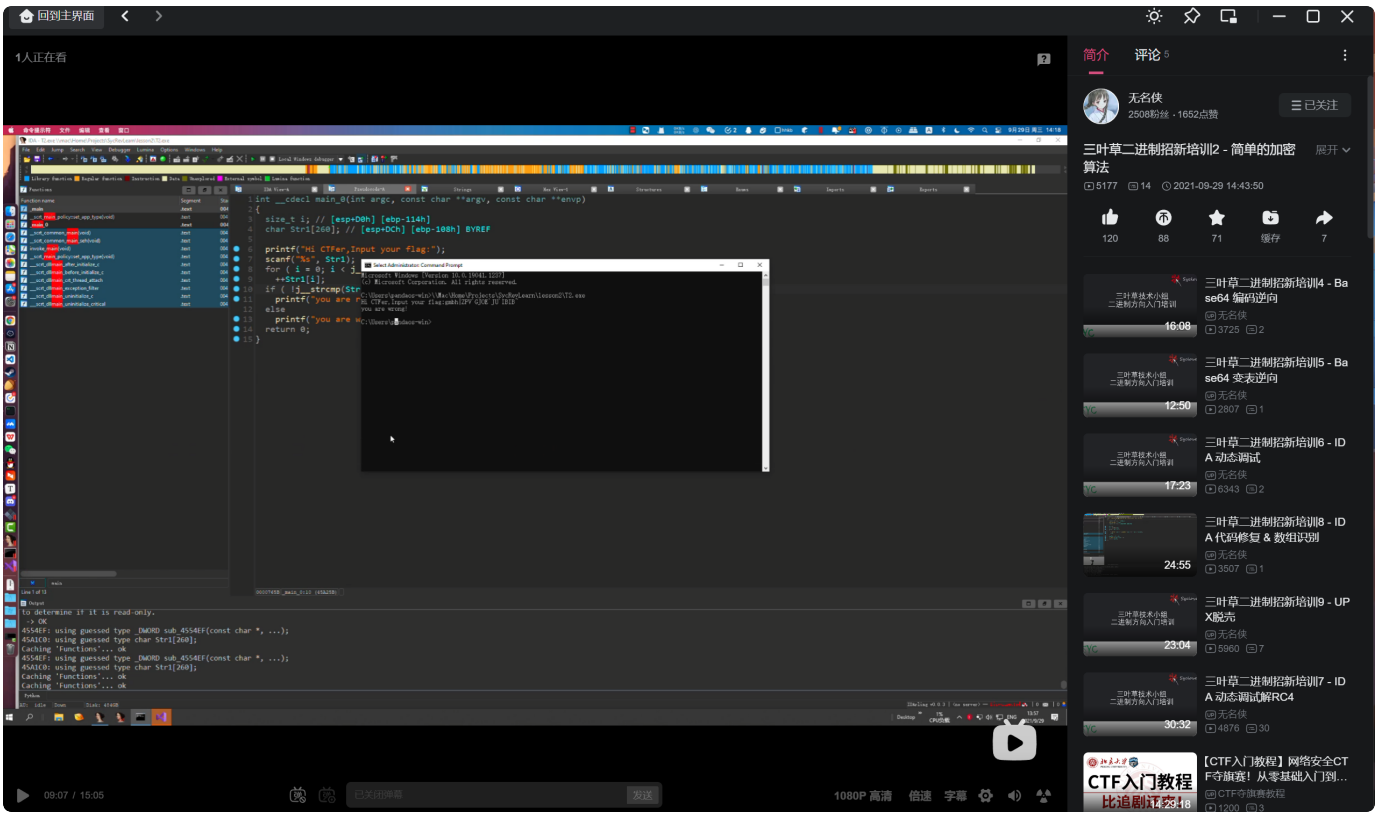
2026/10/24~25

2026/10/16

回顾复习三叶草视频

第一个视频：三叶草二进制招新培训2 – 简单的加密算法

因为已将有些忘了，所以复习一下加深印象



这个是一个简单的加密，类似于凯撒移位，就是给你输入的每个字符都向后加一，然后与代码中给的进行比较，如果一样则正确

视频中ida伪代码加密

```c
scanf("%s", Str1);
for ( i = 0; i < j_strlen(Str1); ++i )
    ++Str1[i];
if ( !j_strlen(Str1,"gmbh|ZPV`GJOE`JU`IBIB~"))
    printf("you are right!\n");
else
    printf("you are wrong!\n");
```

/*因为没写全所以没办法直接在语雀里直接运行*/

现在开始尝试跟着无名侠写解密代码

解密代码

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char Str1[]  = "gmbh|ZPV`GJOE`JU`IBIB~";
    for ( int i = 0; i < strlen(Str1); ++i )
    --Str1[i];
    printf("%s\n", Str1);
    return 0;
}
```



执行成功，解密完成/*感觉挺有成就感，虽然是跟着老师讲的，但也是自己跟着一步步做出来的，学进了脑子里*/

因为以后要用Python写所以再次跟着写一个python的

解密代码

```python
target = bytearray(b'gmbh|ZPV`GJOE`JU`IBIB~')
for i in range (len(target)):
    target[i] -= 1
print(target)
```

我不知道为什么target = bytearray(b'gmbh|ZPV`GJOE`JU`IBIB~')中前面要带一个b，把它去掉试试

```
问题 1  输出  调试控制台  终端  端口

Traceback (most recent call last):
  File "f:\PKW\周报1解密.py", line 1, in <module>
    target = bytearray('gmbh|ZPV`GJOE`JU`IBIB~')
TypeError: string argument without an encoding
PS F:\PKW>
```

报错了，不知道原因，查了查：Python中的bytearray()不能直接接收普通字符串，字符串要转字节，必须指定编码（比如utf-8）/*我的vscode总是代码写的对的，运行出来是乱码因为这个我之前弄了好久*/或者直接传字节串（b"xxx"）

```python
target = bytearray('gmbh|ZPV`GJOE`JU`IBIB~', encoding='utf-8')
for i in range(len(target)):
    target[i] -= 1
print(target.decode('utf-8'))
```

上面的这是另外一种方法/*指定编码（比如utf-8）*/

第二个视频：三叶草二进制招新培训3 – 简单的加密算法2



讲的异或运算，异或出现的频率非常高，它是一个二元运算，这是它的运算符号^。

明文 ^ 密钥 = 密文　　密文 ^ 密钥 = 明文

它的逆运算就是本身

视频中ida查查看的伪代码

```c
printf("[5] Hi CTFer,Input  your f1ag:");
scanf("%s", str1);
for(i = 0; i < j_strlen(Str1); ++i
    Strl[i]^= i;
if( !j_strcmp(Str1, Str2))
    printf("you are right!in");
    else
    printf("you are wrong!n");
return 0;
```

Str2中有一个不可见字符（0x7F）会导致ida中不会将Str2直接识别为字符，所以需要用Str2来表示这一串字符

```
.rdata:00503E73                 db    0
.rdata:00503E74 ; const char aS[]
.rdata:00503E74 aS              db '%s',0               ; DATA XREF: _main_0+3C↑o
.rdata:00503E74                                         ; dump_all_object_since_nolock(_CrtMemState const
.rdata:00503E77                 align 4
.rdata:00503E78 ; const char Str2[4]
.rdata:00503E78 Str2            db 'fmcd'               ; DATA XREF: _main_0:loc_45A261↑o
.rdata:00503E7C                 db  7Fh ;
.rdata:00503E7D                 db  5Ch ; \
.rdata:00503E7E                 db  49h ; I
.rdata:00503E7F                 db  52h ; R
.rdata:00503E80                 db  57h ; W
.rdata:00503E81                 db  4Fh ; O
.rdata:00503E82                 db  43h ; C
.rdata:00503E83                 db  45h ; E
.rdata:00503E84                 db  48h ; H
.rdata:00503E85                 db  52h ; R
.rdata:00503E86                 db  47h ; G
.rdata:00503E87                 db  5Bh ; [
.rdata:00503E88                 db  4Fh ; O
.rdata:00503E89                 db  59h ; Y
.rdata:00503E8A                 db  53h ; S
.rdata:00503E8B                 db  5Bh ; [
.rdata:00503E8C                 db  55h ; U
.rdata:00503E8D                 db  68h ; h
.rdata:00503E8E                 db    0
.rdata:00503E8F                 db    0
.rdata:00503E90                 db    0
```

将这传数据转换成BYTE列表 /*我们要对其中的每一个字节进行操作

跟着无名侠尝试写python解密

```
PS F:\PKW> & D:\python\python.exe f:/PKW/周报1.2解密.py
[102, 108, 97, 103, 123, 89, 107, 120, 84, 100, 117, 87, 69, 95, 89, 64, 83, 84, 90, 65, 83, 78, 89, 78, 75, 66, 79, 115]
```

一开始输出为一串数字，为改为输出为(UTF-8)格式，修改代码成功

周报1.2解密.py > ...

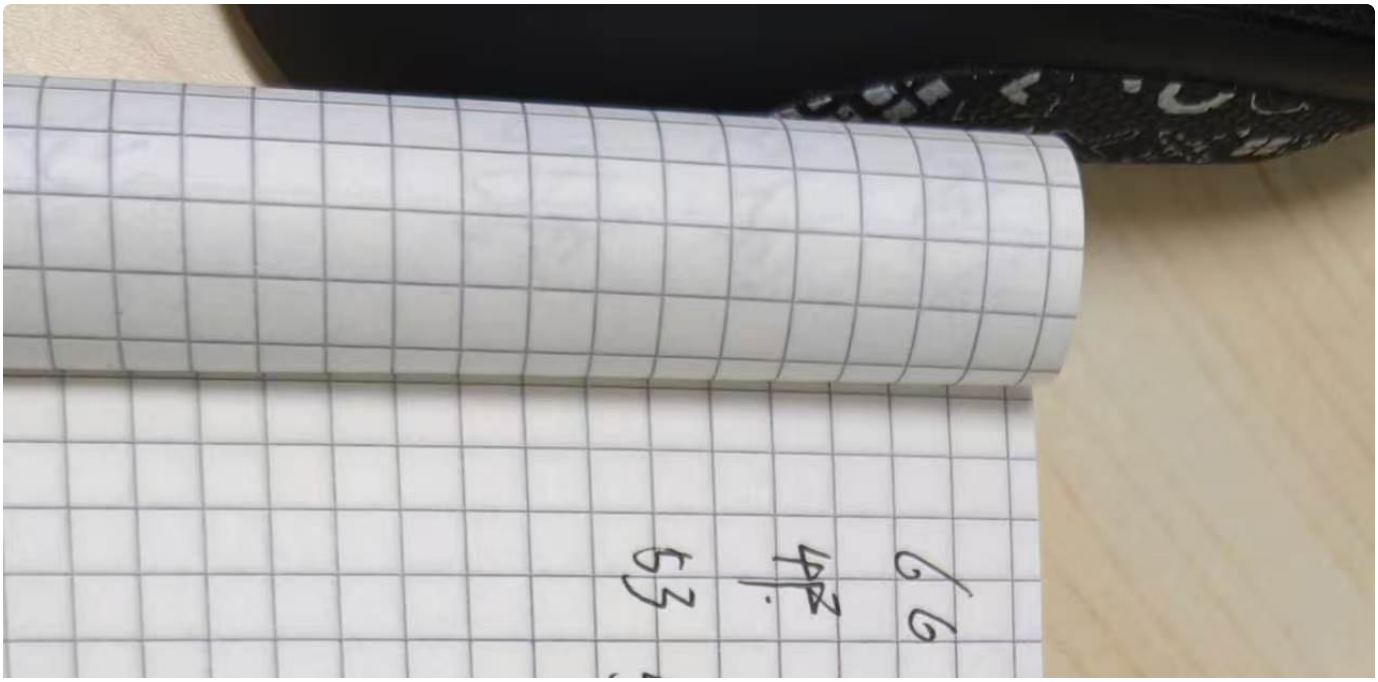
```python
date = [0x66, 0x6D, 0x63, 0x64, 0x7F, 0x5C, 0x6D, 0x7F, 0x5C, 0x6D, 0x7F, 0x5C, 0x49, 0x52
for i in range(len(date)):
    date[i] = date[i] ^ i
date = bytes(date)
print(date.decode('utf-8'))
```

问题 输出 调试控制台 终端 端口

```
AttributeError: 'list' object has no attribute 'decode'
PS F:\PKW> & D:\python\python.exe f:/PKW/周报1.2解密.py
flag{YkxTduWE_Y@STZASNYNKBOs
PS F:\PKW>
```

C/C++ ...
Python
C/C++ ...
Python

因为是纯手打可能有点慢，我没有他的那个题只能将它抄下来



后面是解密代码

解密

```python
date = [0x66, 0x6D, 0x63, 0x64, 0x7F, 0x5C, 0x6D, 0x7F, 0x5C, 0x6D, 0x7F, 0x5C, 0x49, 0x52, 0x57, 0x4F, 0x43, 0x45, 0x48, 0x52, 0x47, 0x5B, 0x4F, 0x59, 0x53, 0x5B, 0x55, 0x68]
for i in range(len(date)):
    date[i] = date[i] ^ i
date = bytes(date)
print(date.decode('utf-8'))
```

```
flag{YkxTduWE_Y@STZASNYNKBOs
```

可以运行，但是少了点字符，可能是我抄错了

打代码时候多打了几个修改完成后运行

修改后

```python
date = [0x66, 0x6D, 0x63, 0x64, 0x7F, 0x5C, 0x49, 0x52, 0x57, 0x4F, 0x43, 0x45, 0x48, 0x52, 0x47, 0x5B, 0x4F, 0x59, 0x53, 0x5B, 0x55, 0x68]
for i in range(len(date)):
    date[i] ^= i
date = bytes(date)
print(date.decode('utf-8'))
```



现在好了

标注/*字符串以0结尾*/

今日学习总结（附录）

虽然只看了两个视频，但主要的是将他们融会贯通(我打字较慢，该练)

Python中的bytearray()不能直接接收普通字符串，字符串要转字节，必须指定编码（比如utf-8）

今天学了些简单的加密的解密，如何写一些简单的解密脚本，如果有不会的要活用ai，今日ai只用来查了一个python的报错，还有一个重中之重的是写代码时要仔细，不然总是容易出错

今天在实验室就先学这些，我回宿舍会再做几道题进行巩固

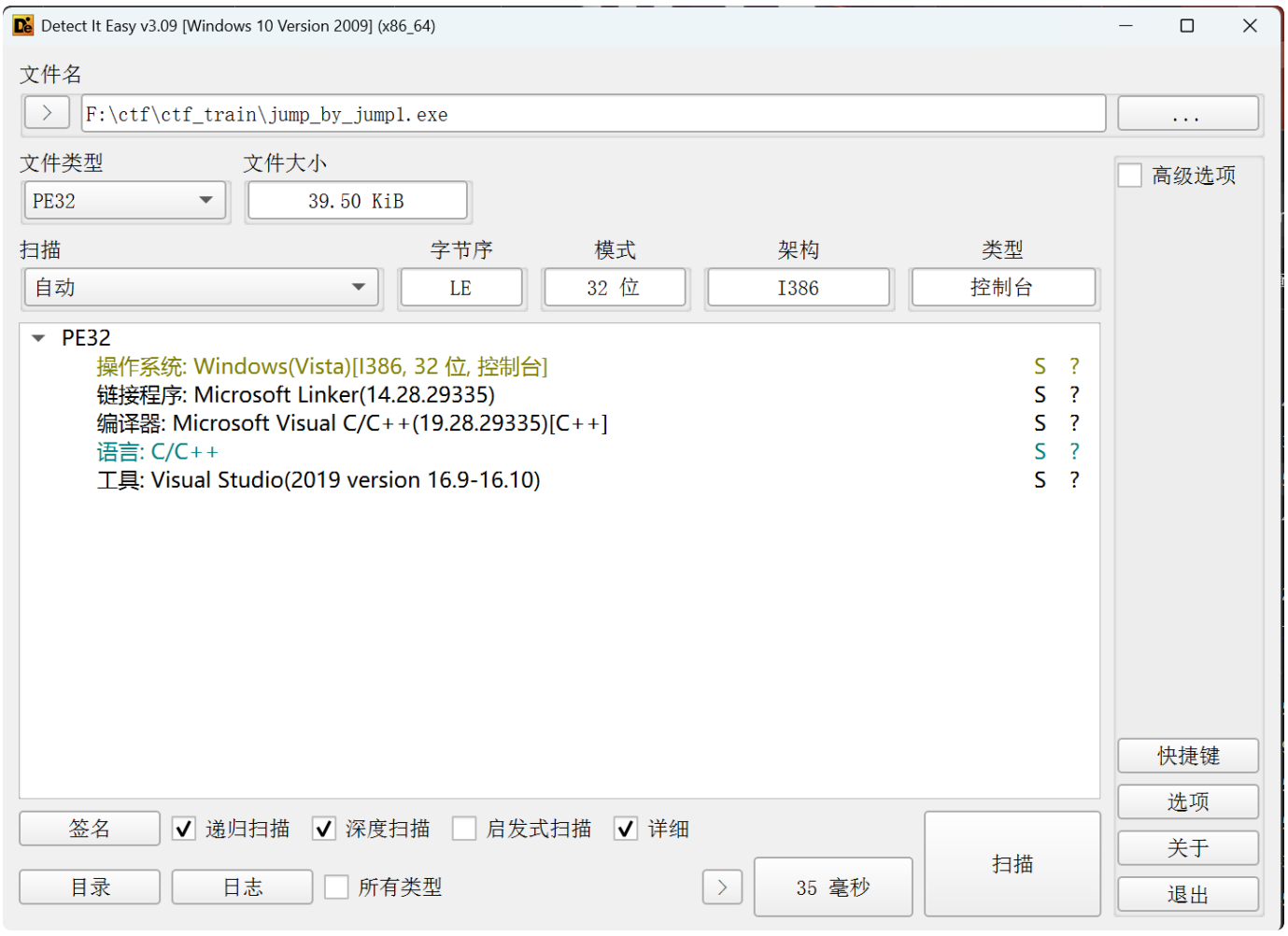
明日目标

因为明天周末，所以可以适当放松，但最起码需要再复习3个视频，并做笔记。抽时间需要做题进行巩固。

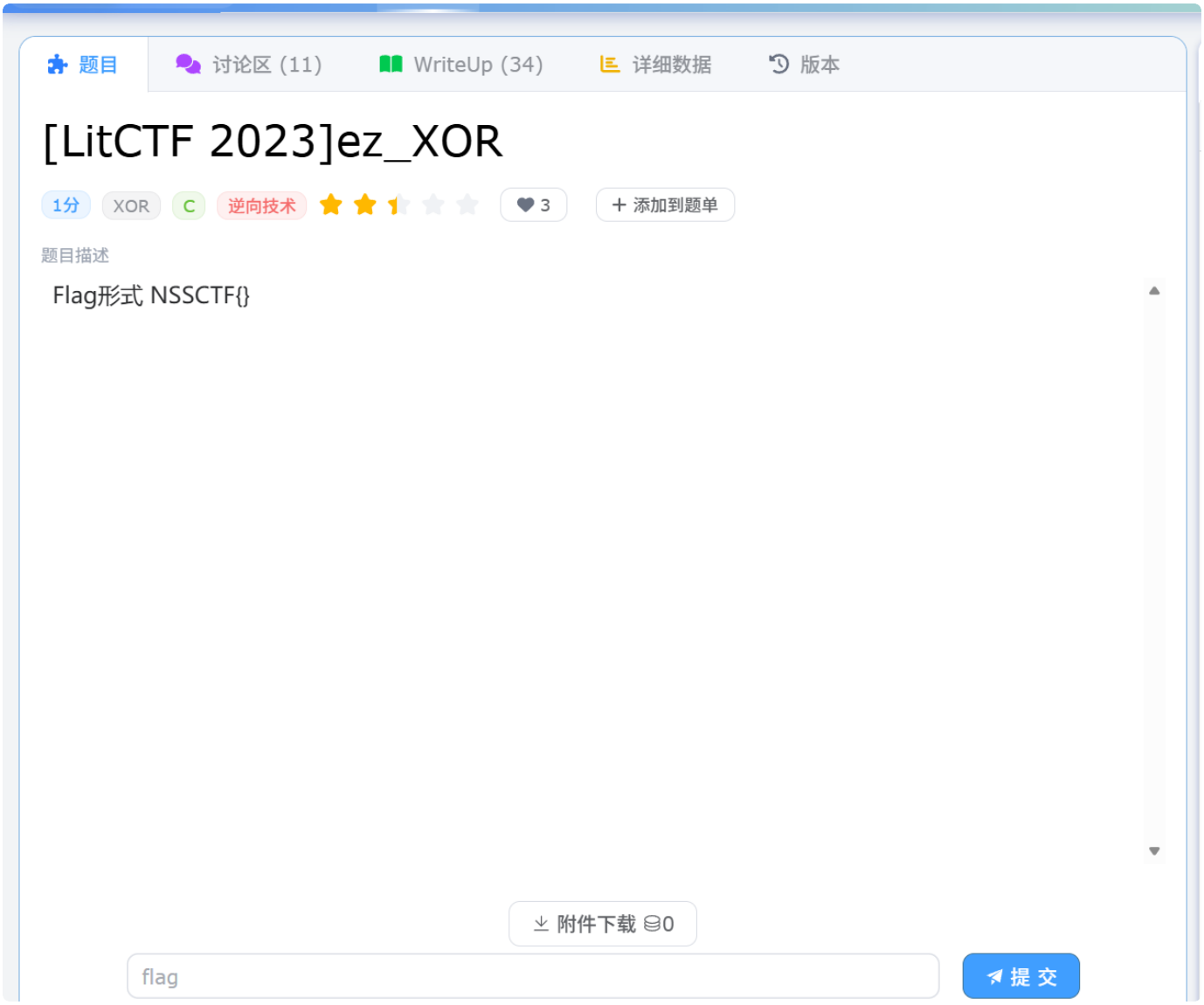
2026/10/17

先做几道题

简单XOR



，发现这道题做过了，做下一道题


题目
讨论区 (11)
WriteUp (34)
详细数据
版本
[LitCTF 2023]ez_XOR
1分
XOR
C
逆向技术
3
+ 添加到题单
题目描述
Flag形式 NSSCTF{}
附件下载 0
flag
提 交

先用DIE查看



PE32位，控制台程序



控制台运行，需要输入正确的flag，用IDA打开/*从程序名字可以看出来这是一道简单的XOR加密*/

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  int argc_1; // [esp+0h] [ebp-80h]
  const char **argv_1; // [esp+4h] [ebp-7Ch]
  const char **envp_1; // [esp+8h] [ebp-78h]
  char Str1[50]; // [esp+1Ch] [ebp-64h] BYREF
  char Str2[26]; // [esp+4Eh] [ebp-32h] BYREF
  __int16 v9; // [esp+68h] [ebp-18h]
  int v10; // [esp+6Ah] [ebp-16h]
  int v11; // [esp+6Eh] [ebp-12h]
  int v12; // [esp+72h] [ebp-Eh]
  int v13; // [esp+76h] [ebp-Ah]
  int v14; // [esp+7Ah] [ebp-6h]
  __int16 v15; // [esp+7Eh] [ebp-2h]

  __main();                             __int16 v15; // [esp+7Eh] [ebp-2h]
  strcpy(Str2, "E`}J]OrQF[V8zV:hzpV}fVF[t");
  v9 = 0;
  v10 = 0;
  v11 = 0;
  v12 = 0;
  v13 = 0;
  v14 = 0;
  v15 = 0;
  printf("Enter The Right FLAG:");
  scanf("%s", Str1);
  XOR(Str1, 3);
  if ( !strcmp(Str1, Str2) )
  {
    printf("U Saved IT!\n");
    return 0;
  }
  else
  {
    printf("Wrong!Try again!\n");
    return main(argc_1, argv_1, envp_1);
  }
}
```

上面是main函数，有明显的XOR，点开查看

Data Unexplored External symbol Lumina function

IDA View-A | Pseudocode-A

```
1  size_t __cdecl XOR(char *Str, char n3)
2  {
3    size_t result; // eax
4    unsigned int i; // [esp+2Ch] [ebp-Ch]
5
6    for ( i = 0; ; ++i )
7    {
8      result = strlen(Str);
9      if ( i >= result )
10       break;
11     Str[i] ^= 3 * n3;
12   }
13   return result;
14 }
```

这个是XOR加密黑盒

写解密代码我不理解为什么vscode要让我

```
char *plain = cipher;
printf("解密结果: %s\n", plain);
return 0;
```

在plain前加一个*

XOR

```c
#include <stdio.h>
#include <string.h>

int main()  /*变量名：cipher(密文)，key(密钥)，plain(明文)*/
{
    char cipher[] = "E`}1]OrQF[V8zV:hzpV}fVF[t+";   /*IDA中看到的密文*/

    char key = 3 * 3;  /*key = 3*n3 n3 = 3*/

    for (int i = 0; cipher[i] != '\0'; i++) {
        cipher[i] ^= key;
    }
    char *plain = cipher;   /*这里不理解为什么有一个‘*’*/
    printf("解密结果：%s\n", plain);
    return 0;
}
```

查了一下

> `char *plain` 里的 `*` 表示 `plain` 是一个**字符指针**，它存放字符数组地址。这里：
>
> char cipher[] = "...";
> char *plain = cipher;
>
> `plain` 指向 `cipher` 的第一个字符。解密循环已经直接修改了 `cipher` 数组，所以 `plain` 指向的内容也就是解密后的字符串， `printf("%s", plain)` 会从该地址开始打印字符，直到遇到 `'\0'` 。

然后运行获得flag

Lit8TF{XOR_1s_3asy_to_OR}

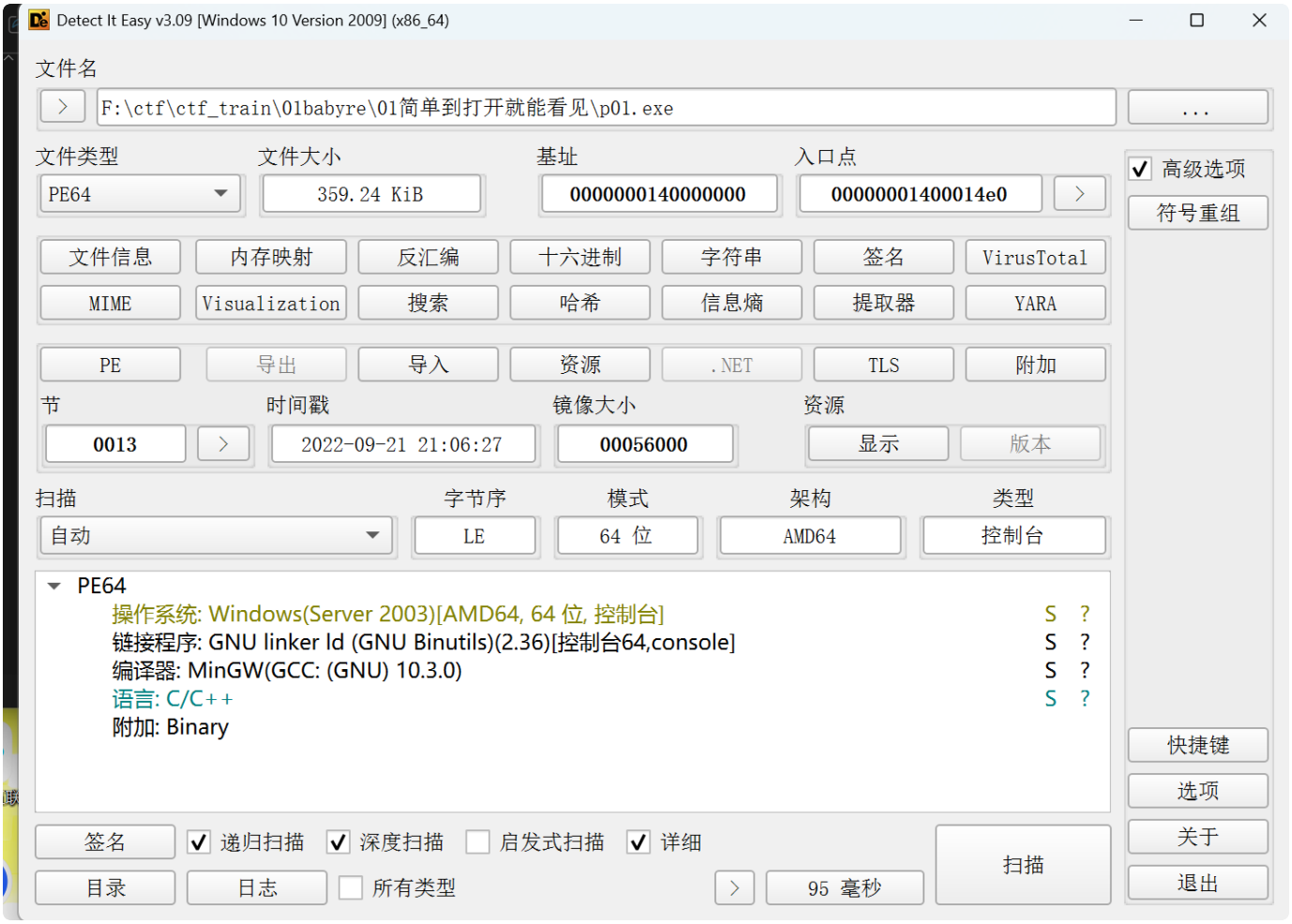


这些是之前做的，python的不想做（不会）

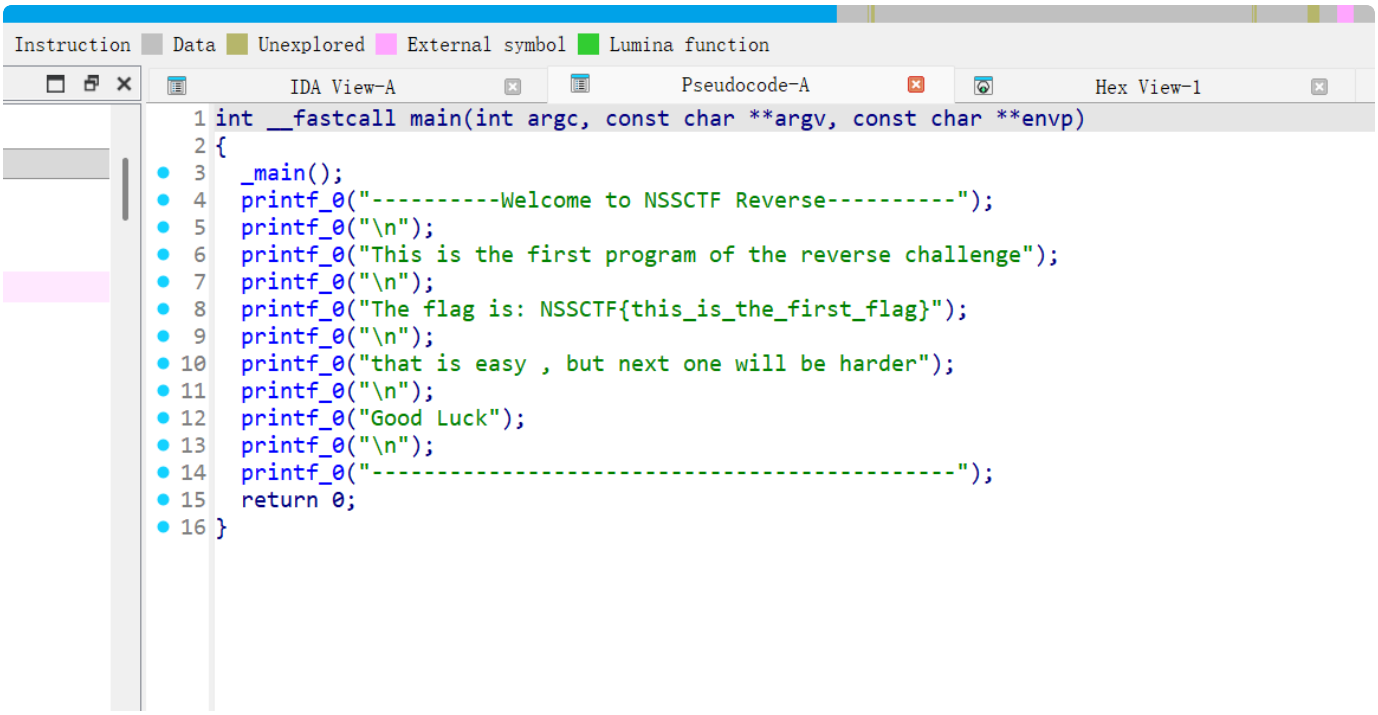
BABBYRE

IDA打开就可以看到

但还是用DIE查看一下

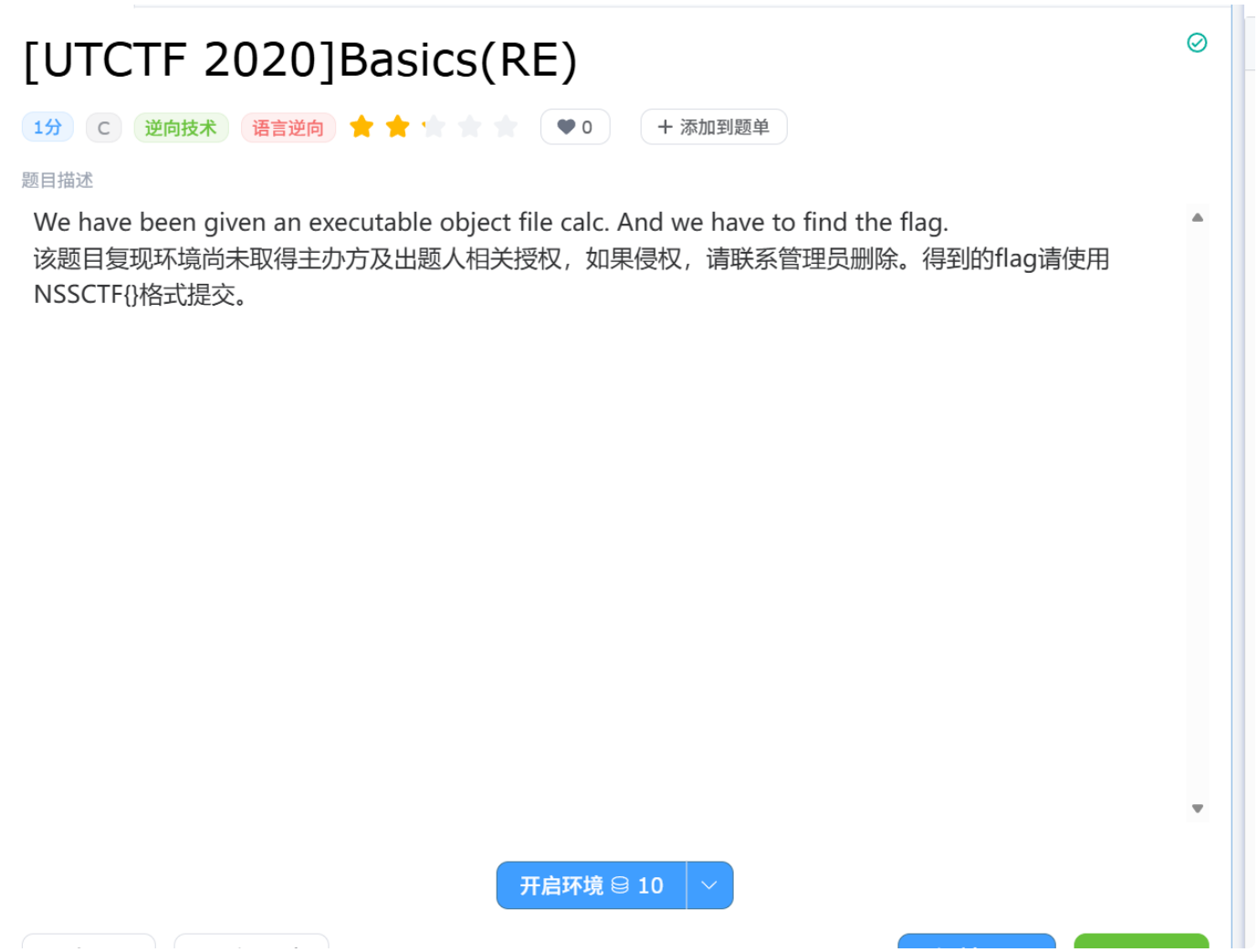


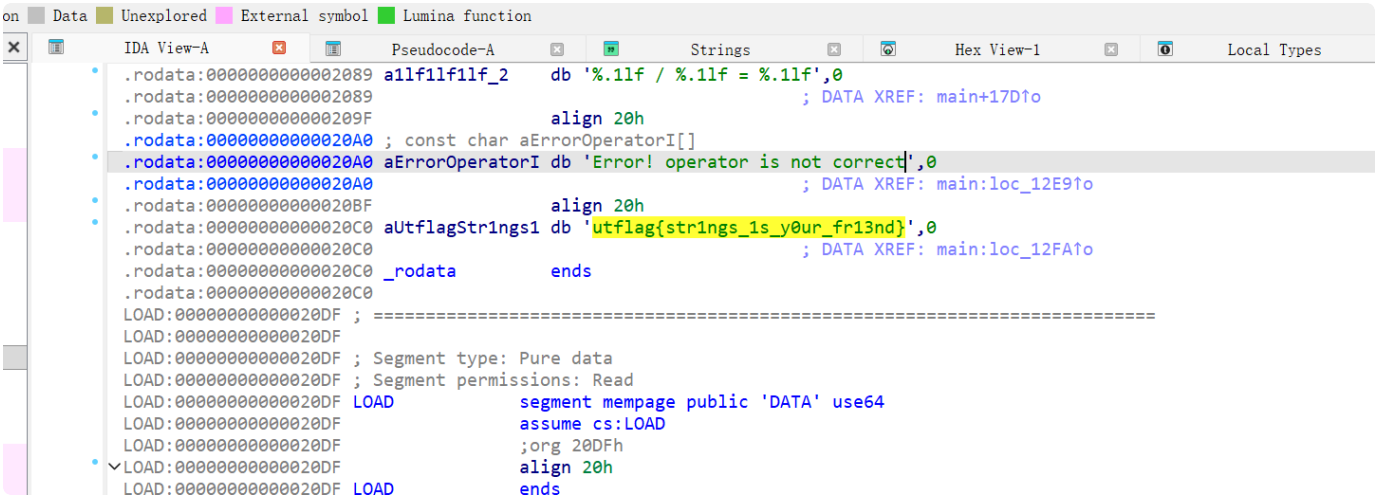
IDA打开直接就可以看到



第三题

也比较简单





是一个linux elf文件IDA打开就可以看到flag

第四题

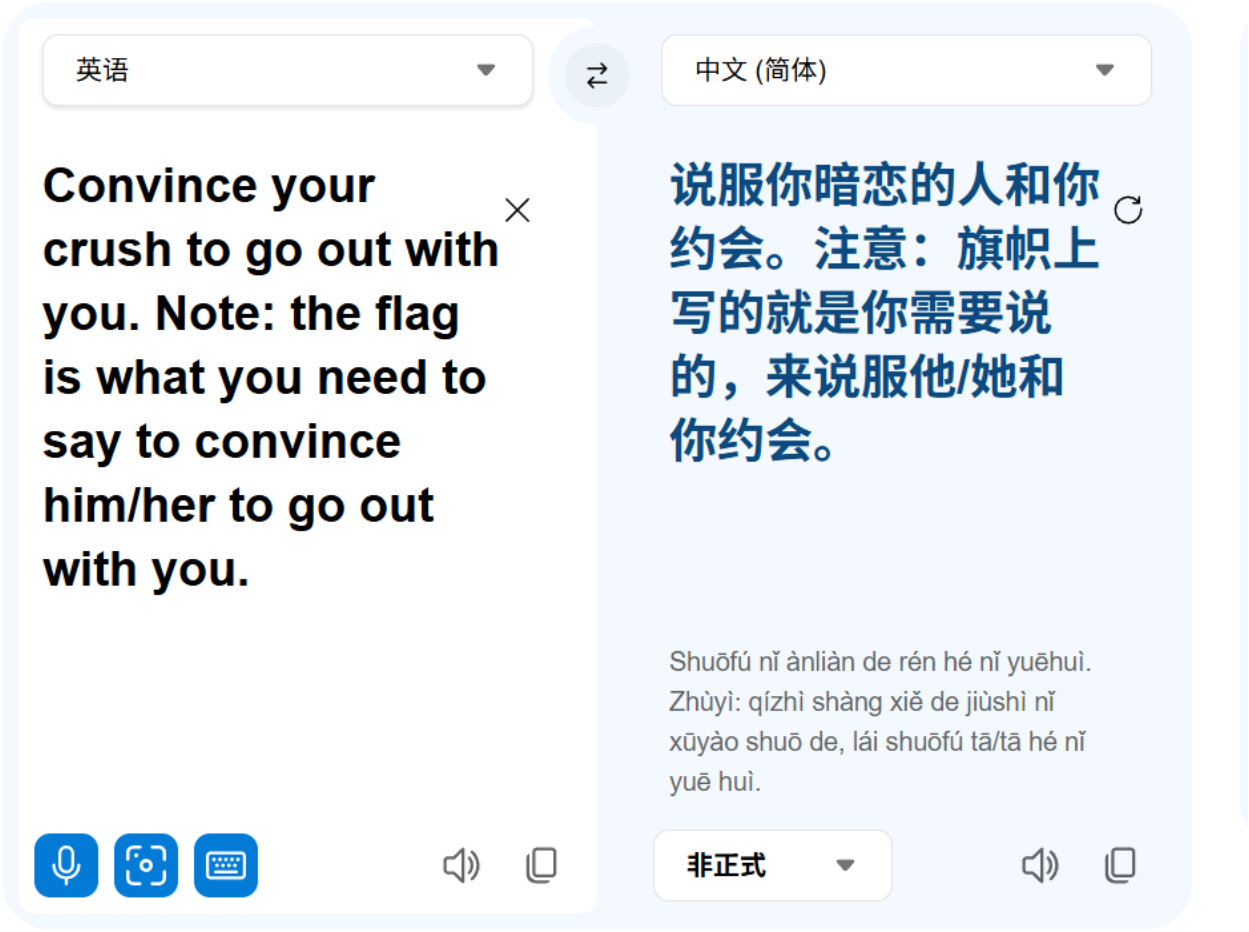
说真的我英语真的很差

[UTCTF 2020]Crack the heart

475分 逆向 ★★★☆☆ 0 + 添加到题单

题目描述

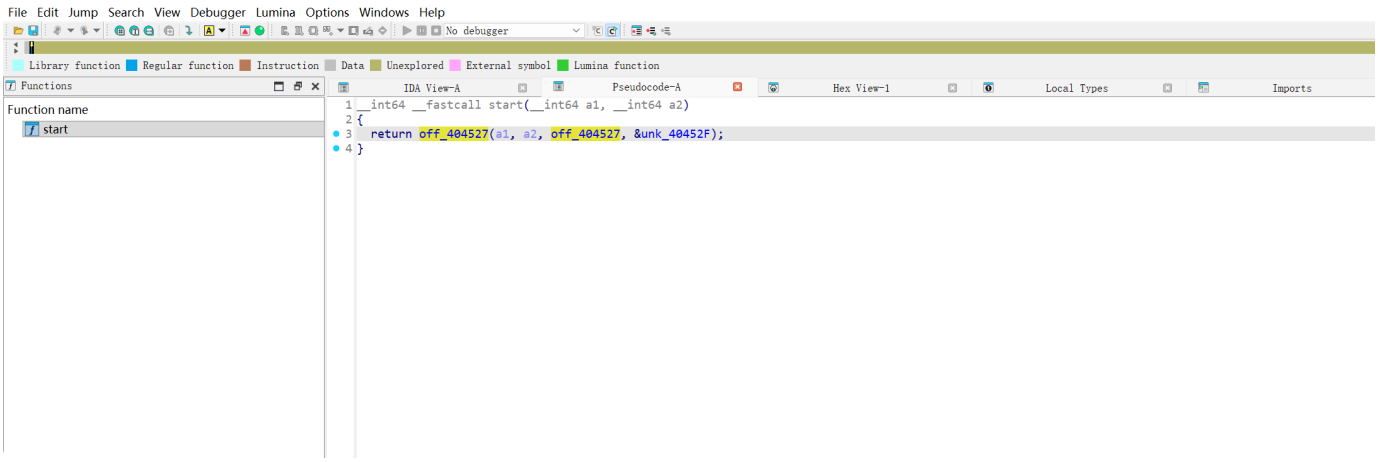
Convince your crush to go out with you. Note: the flag is what you need to say to convince him/her to go out with you.
该题目复现环境尚未取得主办方及出题人相关授权，如果侵权，请联系管理员删除。得到的flag请使用NSSCTF{}格式提交。



DIE查看程序类型，查看之前猜到是ELF程序



无壳，用IDA打开查看



额(⊙﹏⊙)不会下一题

查看wp

EXP

- 具体攻击代码：Angr

```
import angr

proj = angr.Project("./crackme")

simgr = proj.factory.simgr()

simgr.explore(find=lambda s: b"cool" in s.posix.dumps(1))

s = simgr.found[0]

print(s.posix.dumps(0))Angr

Angr was also more efficient than this Python script...

import angr

proj = angr.Project("./crackme")

simgr = proj.factory.simgr()

simgr.explore(find=lambda s: b"cool" in s.posix.dumps(1))

s = simgr.found[0]

print(s.posix.dumps(0))
```

总结

- 对该题的考点总结：sat

> Problem: [UTCTF 2020]Crack the heart

思路

- 解题大致思路：Z3

```
from z3 import *

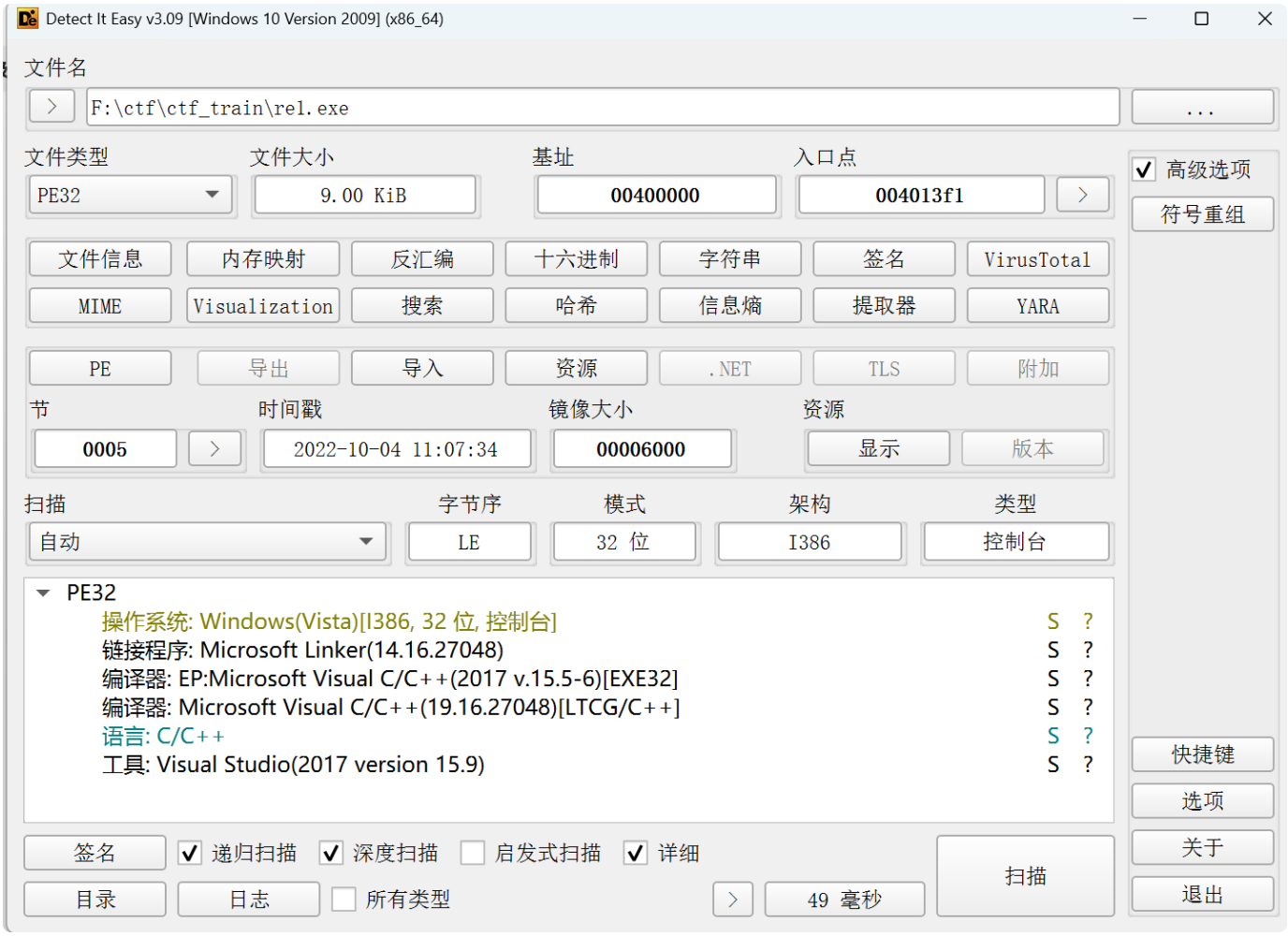
import sys

magics = [
```

看不懂怎莫说，等有时间问问学长/*我就是点了下一题，出现这个，结果不会*/

第五题

简单的花指令，因为之前做过花指令，应该会比较熟悉

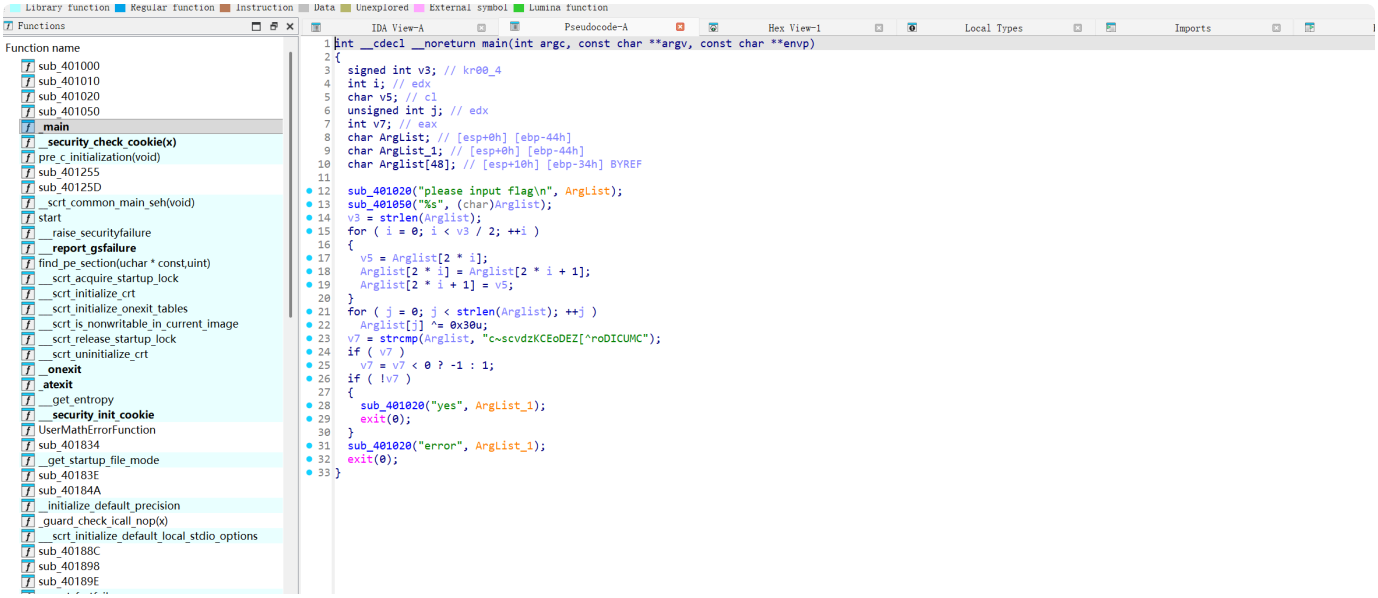



```
.text:004010A3                 push    offset aPleaseInputFla ; "please input flag\n"
.text:004010A8                 call    sub_401020
.text:004010AD                 lea     eax, [ebp-34h]
.text:004010B0                 push    eax
.text:004010B1                 push    offset aS       ; "%s"
.text:004010B6                 call    sub_401050
.text:004010BB                 lea     eax, [ebp-34h]
.text:004010BE                 add     esp, 0Ch
.text:004010C1                 lea     edx, [eax+1]
.text:004010C4
.text:004010C4 loc_4010C4:                             ; CODE XREF: .text:004010C9↓j
.text:004010C4                 mov     cl, [eax]
.text:004010C6                 inc     eax
.text:004010C7                 test    cl, cl
.text:004010C9                 jnz     short loc_4010C4
.text:004010CB                 sub     eax, edx
.text:004010CD                 mov     [ebp-38h], eax
.text:004010D0                 jz      short near ptr loc_4010D4+1
.text:004010D2                 jnz     short near ptr loc_4010D4+1
.text:004010D4
.text:004010D4 loc_4010D4:                             ; CODE XREF: .text:004010D0↑j
.text:004010D4                                         ; .text:004010D2↑j
.text:004010D4                 jmp     near ptr 9A085664H
.text:004010D9 ; ---------------------------------------------------------------------------
.text:004010D9                 sub     eax, edx
.text:004010DB                 xor     edx, edx
.text:004010DD                 mov     esi, eax
.text:004010DF                 sar     esi, 1
.text:004010E1                 test    esi, esi
.text:004010E3                 jle     short loc_401105
.text:004010E5                 db      66h, 66h
.text:004010E5                 nop     word ptr [eax+eax+00000000h]
00000490 00401090: .text:_main (Synchronized with Hex View-1)
```

这就比较简单了

这是一个互补条件加一个垃圾指令的花指令

去除掉之后main就会出现/*要定义函数（按P）*/



这是一个XOR加密，参照上一个XOR加密写脚本

还是C语言脚本吧，简单点

直接套用上一个，改一些代码根据伪代码

```c
#include <stdio.h>
#include <string.h>

int main()  /*变量名：cipher(密文)，key(密钥)，plain(明文)*/
{
    char cipher[] = "c~scvdzKCEoDEZ[^roDICUMC";   /*IDA中看到的密文*/

    char key = 0x30u;  /*key = 0x30*/

    for (int i = 0; i < strlen(cipher) - 1; i++) {
    cipher[i] ^= key;
}
    for (int i = 0; i < strlen(cipher) - 1; i += 2) {
        char temp = cipher[i];
        cipher[i] = cipher[i + 1];
        cipher[i + 1] = temp;
    }


    char *plain = cipher;   /*这里不理解为什么有一个‘*’*/
    printf("解密结果：%s\n", plain);
    return 0;
}
```

这是写的第一个，提交flag错误，不知道为啥，问问AI

strlen() 本来就不包含结尾的 '\0'，不需要再减一。 sizeof(cipher) – 1 才需要减一，这两种计算方式要区分。

> /*主要是我写了 – 1之后密文最后一个C没有被解密，所以flag最后会出现一个C而不是s（'C' ^ 0x30 = 's'）经过交换之后，到了倒数第二个位置，漏掉了这次 XOR，之后它又被交换到倒数第二个位置，所以出现了 BytesC}，而不是 Bytess}*/

修改之后

代码

修改之后

```c
#include <stdio.h>
#include <string.h>

int main()  /*变量名：cipher(密文)，key(密钥)，plain(明文)*/
{
    char cipher[] = "c~scvdzKCEoDEZ[^roDICUMC";   /*IDA中看到的密文*/

    char key = 0x30u;  /*key = 0x30*/

   for (int i = 0; i < strlen(cipher); i++) {
    cipher[i] ^= key;
}
    for (int i = 0; i < strlen(cipher) - 1; i += 2) {
        char temp = cipher[i];
        cipher[i] = cipher[i + 1];
        cipher[i + 1] = temp;
    }



    char *plain = cipher;   /*这里不理解为什么有一个‘*’*/
    printf("解密结果：%s\n", plain);
    return 0;
}
```

获得flag

解密结果：NSSCTF{Just_junk_Bytess}

结语

今天就先到这



做了8道题，在这里写了5道，那三道忘写了，不想再做一遍都是简单的。明天应抱有更大的热情和动力

附录：

strlen() 本来就不包含结尾的 '\0'，不需要再减一。 sizeof(cipher) – 1 才需要减一，这两种计算方式要区分。

`char *plain` 里的 `*` 表示 `plain` 是一个**字符指针**，它存放字符数组地址。这里：

char cipher[] = "...";
char *plain = cipher;

`plain` 指向 `cipher` 的第一个字符。解密循环已经直接修改了 `cipher` 数组，所以 `plain` 指向的内容也就是解密后的字符串， `printf("%s", plain)` 会从该地址开始打印字符，直到遇到 `'\0'` 。

然后运行获得flag

2026/10/18（10/19进行补充）

base64

> 简单来说，Base64编码是一种将二进制数据转换为ASCII字符的技术，为了确保数据在网络上传输时的完整性。Base64通过将每三个字节的数据转换位4个6位的字符，并对不足的数据进行补零处理，使用特定字符集进行编码。编码完成后，如果数据长度不是24位的倍数，会用"="进行填充。解码过程则是编码的逆过程。
>
> Base64编码的设计初衷是为了在数据传输过程中，通过特定的字符集编码方式，确保数据的安全性 和完整性。当原始数据包含无法通过某些媒介传输的二进制字节时，Base64提供了一种字符映射方案，使得这些字节能够转换为可通过文本协议传输的ASCII字符

应用场景

1. 电子邮箱传输：在邮件中传输二进制数据时，如附件或图片。
2. 网页编码 ： 将非文本数据嵌入到HTML或CSS中。
3. 编程语言序列化 ： 在某些语言中，如Python，用于对象序列化。
4. 数据存储 ： 存储不需要二进制处理的数据，如数据库中的文本字段。

Base64编码是数据传输中的一种常用编码格式，主要用于将非ASCII字符的数据转换为ASCII字符集中的字符，这样可以保证数据的兼容性和避免传输过程中的错误。本章将深入探讨Base64编码的原理和步

骤。

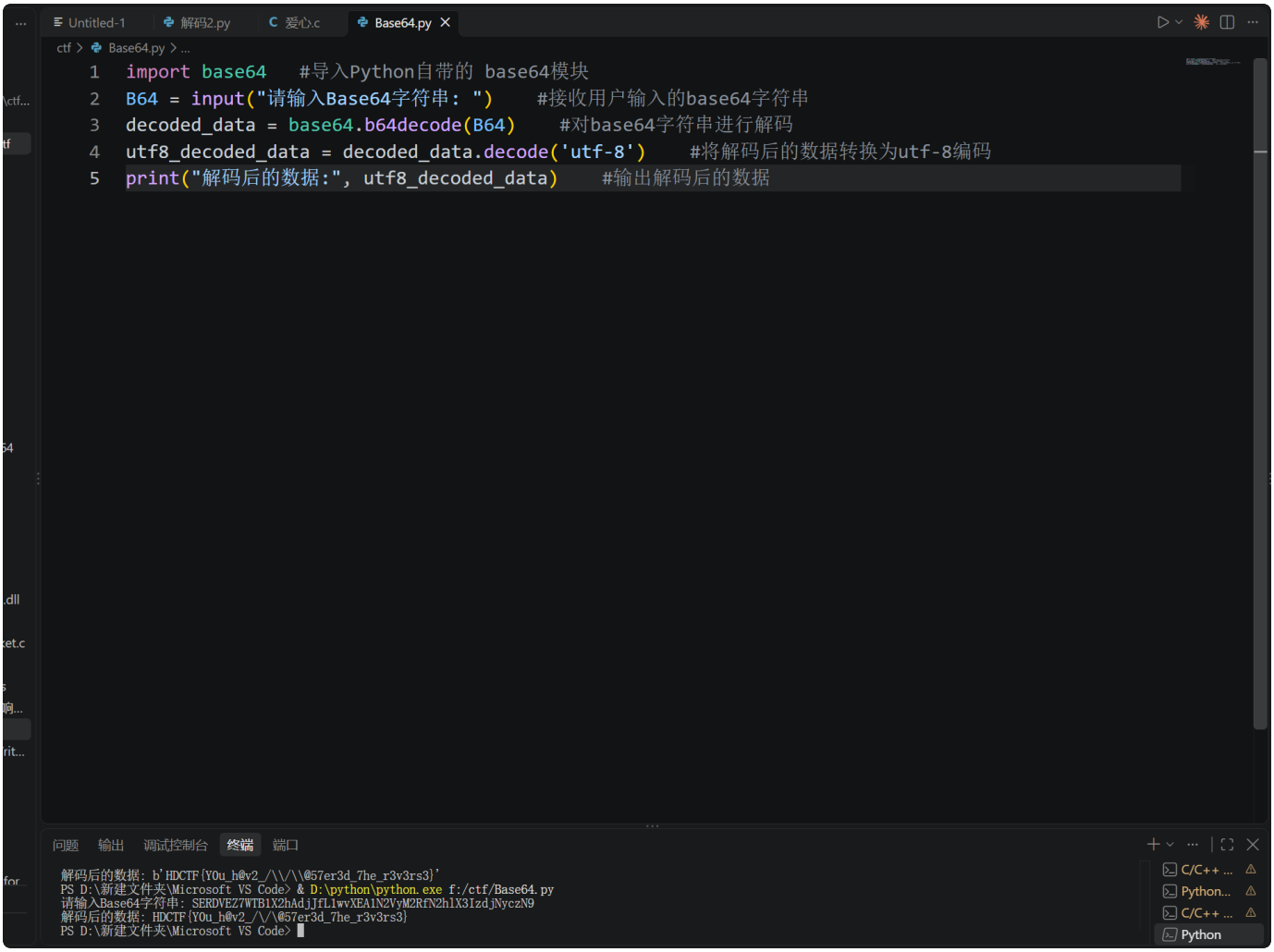
原理：Base64编码的核心思想是将二进制数据分成长度为3的块，然后将每块转换成4个字符表示。具体步骤如下：

1. 将原始数据按 **3 字节（3×8=24位）** 为一组。
2. 将 24 位数据按 **6 位** 分成 4 组（4×6=24位）。
3. 每组 6 位转换为十进制索引，对应索引表中的字符。
4. 若原数据不足 3 字节，用 *0* 补齐，再用 = 作为填充符。



解码的过程：查索引表得到 6 位二进制 → 合并成 8 位字节 → 去掉补位。

py脚本　　#跟着博客写的里面的备注是我自己的理解

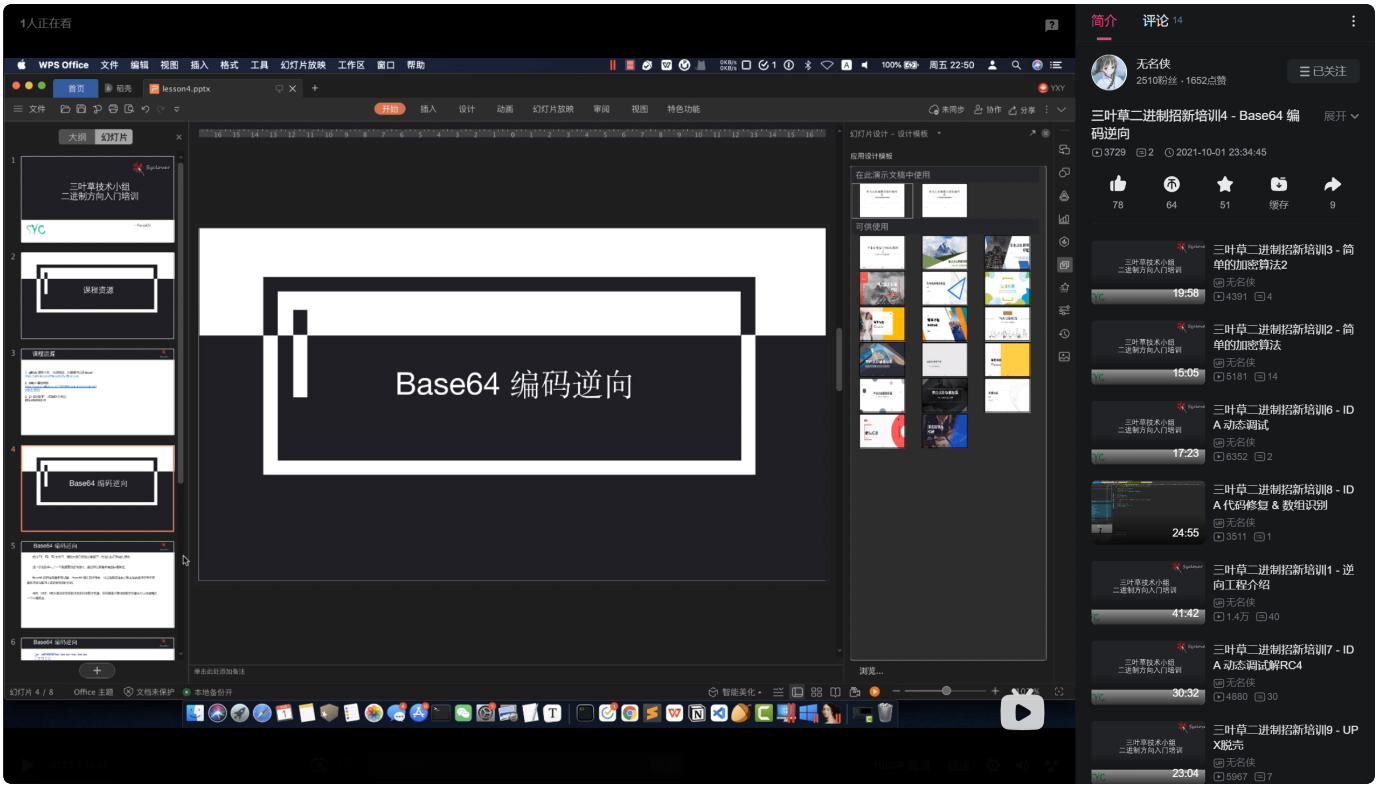


base64 Python

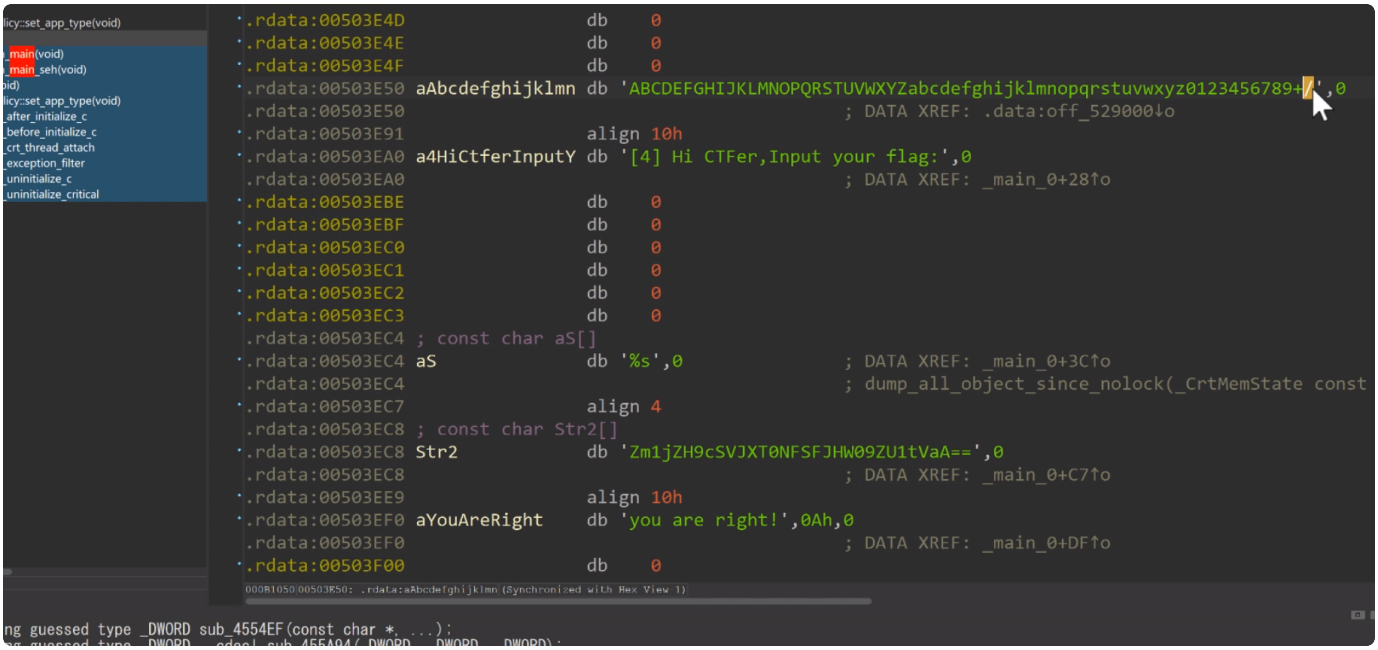
```python
import base64   #导入Python自带的 base64模块
B64 = input("请输入Base64字符串: ")    #接收用户输入的base64字符串
decoded_data = base64.b64decode(B64)    #对base64字符串进行解码
utf8_decoded_data = decoded_data.decode('utf-8')    #将解码后的数据转换为utf-8编码
print("解码后的数据:", utf8_decoded_data)    #输出解码后的数据
```

补零：指在二进制数据序列中添加额外的0位，将其长度调整为满足编码算法要求的形式。编码数据长度必须是3的倍数（因为Base64将每3个字节（24位）的数据转换为4个Base64字符）

突然发现三叶草讲base64，开始听



遇到没有见过的代码可以通过常量来查找





简便点理解，比如PKW这三个字母P（80）K（75）W（87）写成二进制：01010000 01001011 01010111

切成四组，每组6b，010100 000100 101101 010111分别对应U E t X，用脚本检验一下

运行结果

```
请输入Base64字符串: UEtX
解码后的数据: PKW

=== 代码执行成功 ===
```

没问题

我想一个长点的，比如一个20字节的明文，bit = 20*8=160，每六个一切，160/6=6余下4bit，加两个0，凑够6bit，总共27组，base64必须是4的整数倍，所以结尾要补一个填充=，/*我有一个疑问为什么要是6bit一拆，二进制111111转换过来是“/”所以刚好为最后一个63，64对应0~63*/将上面27个全部转换过来，最后面加一个等号补充，这就是base64编码

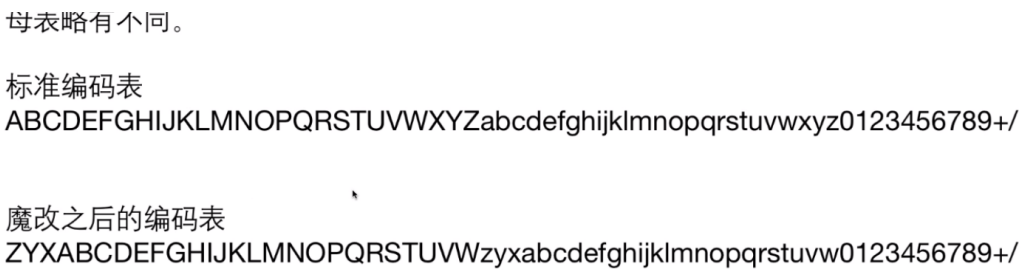
写脚本的话，需要先将明文转换为

自定义base64编码，在知道明文和密文的前提下，推编码表：

然后我猜测，一个已知明文和密文，先将明文转为ASCII值，然后转为二进制排列，每6bit一组排列，如果不够进行补0，然后把密文也转为ASCII值，然后转为二进制排列，6bit一组，一一对照进行补充编码表，emm，补充=要去掉

Base64变表逆向

其基本原理是不变,只是改变了它的索引表，正如变表这个名字说的一样



这种就是改变了编码表

RC4

RC4不是分组密码，是流密码，是根据密钥生成一串伪随机字节，称为密钥流，再把密钥流与明文逐字节异或

即

密文 = 明文 XOR 密钥流

明文 = 密文 XOR 密钥流

RC4是一个对等算法，加密的本身就是它的解密，所以解密就可以在同样的代码里进行解密

只需要进行修改它的输入和输出

RC4加密

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    unsigned char key[] = "key";
    unsigned char input[] = "Plaintext";
    unsigned char output[sizeof(input) - 1];

    int key_len = (int)strlen((char *)key);
    int len = (int)sizeof(input) - 1;
    unsigned char S[256];
    int i, j, n;
    unsigned char temp;

    for (i = 0; i < 256; i++)
        S[i] = (unsigned char)i;

    j = 0;
    for (i = 0; i < 256; i++) {
        j = (j + S[i] + key[i % key_len]) & 0xff;
        temp = S[i];
        S[i] = S[j];
        S[j] = temp;
    }

    i = 0;
    j = 0;
    for (n = 0; n < len; n++) {
        i = (i + 1) & 0xff;
        j = (j + S[i]) & 0xff;

        temp = S[i];
        S[i] = S[j];
        S[j] = temp;

        output[n] = input[n] ^ S[(S[i] + S[j]) & 0xff];
    }

    for (n = 0; n < len; n++)
        printf("%02X", output[n]);
    printf("\n");

    return 0;
}
```

对应的解密

对应的解密

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    unsigned char key[] = "key";
    unsigned char input[] = {
        0x5B, 0x00, 0x55, 0x84, 0x4A,
        0xFB, 0x1E, 0x32, 0x3C
    };
    unsigned char output[sizeof(input)];

    int key_len = (int)strlen((char *)key);
    int len = (int)sizeof(input);
    unsigned char S[256];
    unsigned char temp;
    int i, j, n;

    for (i = 0; i < 256; i++)
        S[i] = (unsigned char)i;

    j = 0;
    for (i = 0; i < 256; i++) {
        j = (j + S[i] + key[i % key_len]) & 0xff;

        temp = S[i];
        S[i] = S[j];
        S[j] = temp;
    }

    i = 0;
    j = 0;
    for (n = 0; n < len; n++) {
        i = (i + 1) & 0xff;
        j = (j + S[i]) & 0xff;

        temp = S[i];
        S[i] = S[j];
        S[j] = temp;

        output[n] = input[n] ^ S[(S[i] + S[j]) & 0xff];
    }

    /* 若明文是文本，按字符串输出 */
    printf("%.*s\n", len, output);
```

```
46
47          return 0;
48      }
```

TEA家族（10/22）

TEA

分组大小64bit（2个字），密钥128bit（4个字）进行加密32轮，密钥固定按从头到尾进行即k0–>k3

存在的缺陷：

1. 等价密钥：TEA名义上有2^{128}个密钥，实际上每4个密钥加密效果完全一样　　，有效密钥只有2^{126}。（k0/k1和k2/k3有对称性质）
2. 密钥表太规律：每轮用的都是相同的k0 k1 k2 k3

TEA加密

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define DELTA  0x9E3779B9u
#define ROUNDS 32

void tea_encrypt(uint32_t v[2], const uint32_t k[4])
{
    uint32_t v0 = v[0], v1 = v[1];
    uint32_t sum = 0;

    for (int i = 0; i < ROUNDS; i++) {
        sum += DELTA;                                   /* 每轮先加 delta */
        v0 += ((v1 << 4) + k[0]) ^ (v1 + sum) ^ ((v1 >> 5) + k[1]);  /* 用 v1 更新 v0 */
        v1 += ((v0 << 4) + k[2]) ^ (v0 + sum) ^ ((v0 >> 5) + k[3]);  /* 用新的 v0 更新 v1 */
    }

    v[0] = v0;
    v[1] = v1;
}

int main(void)
{
    char msg[] = "flag{tea_is_easy}";        /* 17 字节（不含结尾 '\0'） */
    uint32_t key[4] = {0x01234567, 0x89ABCDEF, 0xFEDCBA98, 0x76543210};

    int len    = (int)strlen(msg);
    int blocks = (len + 7) / 8;              /* 向上取整，17 -> 3 块 */

    printf("明文: %s (%d 字节，补 0 到 %d 字节)\n", msg, len, blocks * 8);

    for (int i = 0; i < blocks; i++) {
        uint32_t v[2] = {0, 0};              /* 先清零，不足 8 字节的部分自然就是 0 */
        int n = len - i * 8;                 /* 这一块还剩几个字节 */
        if (n > 8) n = 8;

        memcpy(v, msg + i * 8, n);           /* 只拷实际存在的字节 */
        tea_encrypt(v, key);

        printf("第 %d 块密文: %08X %08X\n", i + 1, v[0], v[1]);
```

```
42        }
43
44        return 0;
45    }
```

我让DSH生成的加密代码

解密过程就是将它倒推

首先第11行sum = DELTA * ROUNDS（加密时，是正着，这一行是让它从终点出发）

TEA为对称加密，密钥必须一模一样，所以

```c
uint32_t key[4] = {0x01234567, 0x89ABCDEF, 0xFEDCBA98, 0x76543210};
```

加密时解出来

```
明文: flag{tea_is_easy}（17 字节，补 0 到 24 字节）
第 1 块密文: 047C51F7 8AABF213
第 2 块密文: 84DE4A65 7515C91C
第 3 块密文: A6E279E6 E64E1A34
```

```c
uint32_t cipher[3][2] = {
    {0x047C51F7, 0x8AABF213},
    {0x84DE4A65, 0x7515C91C},
    {0xA6E279E6, 0xE64E1A34},
};
```

密文是按块存放，所以有3块

int blocks = 3;

解密

解密

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define DELTA  0x9E3779B9u
#define ROUNDS 32

void tea_decrypt(uint32_t v[2], const uint32_t k[4])
{
    uint32_t v0 = v[0], v1 = v[1];
    uint32_t sum = DELTA * ROUNDS;              /* 加密结束时的 sum */

    for (int i = 0; i < ROUNDS; i++) {
        v1 -= ((v0 << 4) + k[2]) ^ (v0 + sum) ^ ((v0 >> 5) + k[3]);  /* 先撤 v1 */
        v0 -= ((v1 << 4) + k[0]) ^ (v1 + sum) ^ ((v1 >> 5) + k[1]);  /* 再撤 v0 */
        sum -= DELTA;                                                /* sum 往回退 */
    }

    v[0] = v0;
    v[1] = v1;
}

int main(void)
{
    uint32_t key[4] = {0x01234567, 0x89ABCDEF, 0xFEDCBA98, 0x76543210};

    /* 3 块密文，就是 tea_str_enc 打印出来的值 */
    uint32_t cipher[3][2] = {
        {0x047C51F7, 0x8AABF213},
        {0x84DE4A65, 0x7515C91C},
        {0xA6E279E6, 0xE64E1A34},
    };
    int blocks = 3;                             /*有3块*/

    unsigned char out[25];                      /* 24 字节明文 + 1 个 '\0' */

    for (int i = 0; i < blocks; i++) {
        uint32_t v[2] = {cipher[i][0], cipher[i][1]};
        tea_decrypt(v, key);                    /* 解密这一块 */
        memcpy(out + i * 8, v, 8);              /* 8 字节写回结果缓冲区 */
    }
```

```
    out[24] = '\0';                                 /* 补字符串结尾 */

    printf("明文: %s\n", out);                      /* flag{tea_is_easy} */
    return 0;
}
```

BlockTEA

分组大小变大， 密钥128bit（4个字）加密长度与其长度有关，可以被明文攻击破解

XTEA

分组大小64bit（2个字），密钥128bit（4个字）进行加密32轮（64个半轮），与TEA区别，没有规律性，不存在等价密钥

不再使用固定k0~k3，而使用sum的值动态挑一个密钥字

XXTEA

分组大小变长（n>=2个字），密钥128bit（4个字）加密轮数为6 + 52/n，整段数据连锁加密

不再8字节为一组，而是把整条消息当成一个大整体，让每个字都受前后字影响

2026/10/19

IDA动态调试



视频中有一个函数qmemcpy是内存复制

```
size_t i; // [esp+190h] [ebp-340h]
char input[264]; // [esp+3A4h] [ebp-12Ch] BYREF
char Str1[32]; // [esp+4ACh] [ebp-24h] BYREF

qmemcpy(Str1, &unk_50DE50, 0x1Cu);
printf("[6] Hi CTFer,Input your flag:");
scanf("%s", input);
for ( i = 0; i < j__strlen(Str1); ++i )
  Str1[i] = ((i + 1) ^ Str1[i]) - i;
```

将第二个参数所对应的值复制28个字节到Str1参数中

```
.rdata:0050DE4D                 db    0
.rdata:0050DE4E                 db    0
.rdata:0050DE4F                 db    0
.rdata:0050DE50 unk_50DE50      db  47h ; G             ; DATA XREF: _main_0+2D↑o
.rdata:0050DE51                 db  6Fh ; o
.rdata:0050DE52                 db  60h ; `
.rdata:0050DE53                 db  6Eh ; n
.rdata:0050DE54                 db  7Ah ; z
.rdata:0050DE55                 db  5Fh ; _
.rdata:0050DE56                 db  69h ; i
.rdata:0050DE57                 db  78h ; x
.rdata:0050DE58                 db  72h ; r
.rdata:0050DE59                 db  62h ; b
.rdata:0050DE5A                 db  58h ; X
.rdata:0050DE5B                 db  52h ; R
.rdata:0050DE5C                 db  66h ; f
.rdata:0050DE5D                 db  6Fh ; o
.rdata:0050DE5E                 db  4Ah ; J
.rdata:0050DE5F                 db  7Eh ; ~
.rdata:0050DE60                 db  45h ; E
.rdata:0050DE61                 db  44h ; D
.rdata:0050DE62                 db  47h ; G
.rdata:0050DE63                 db  7Ch ; |
.rdata:0050DE64                 db  4Eh ; N
.rdata:0050DE65                 db  62h ; b
.rdata:0050DE66                 db  4Ch ; L
.rdata:0050DE67                 db  40h ; @
.rdata:0050DE68                 db  72h ; r
.rdata:0050DE69                 db  68h ; h
.rdata:0050DE6A                 db  8Ch
.rdata:0050DE6B                 db    0
```

存在一些不可见字符，最后以0结尾

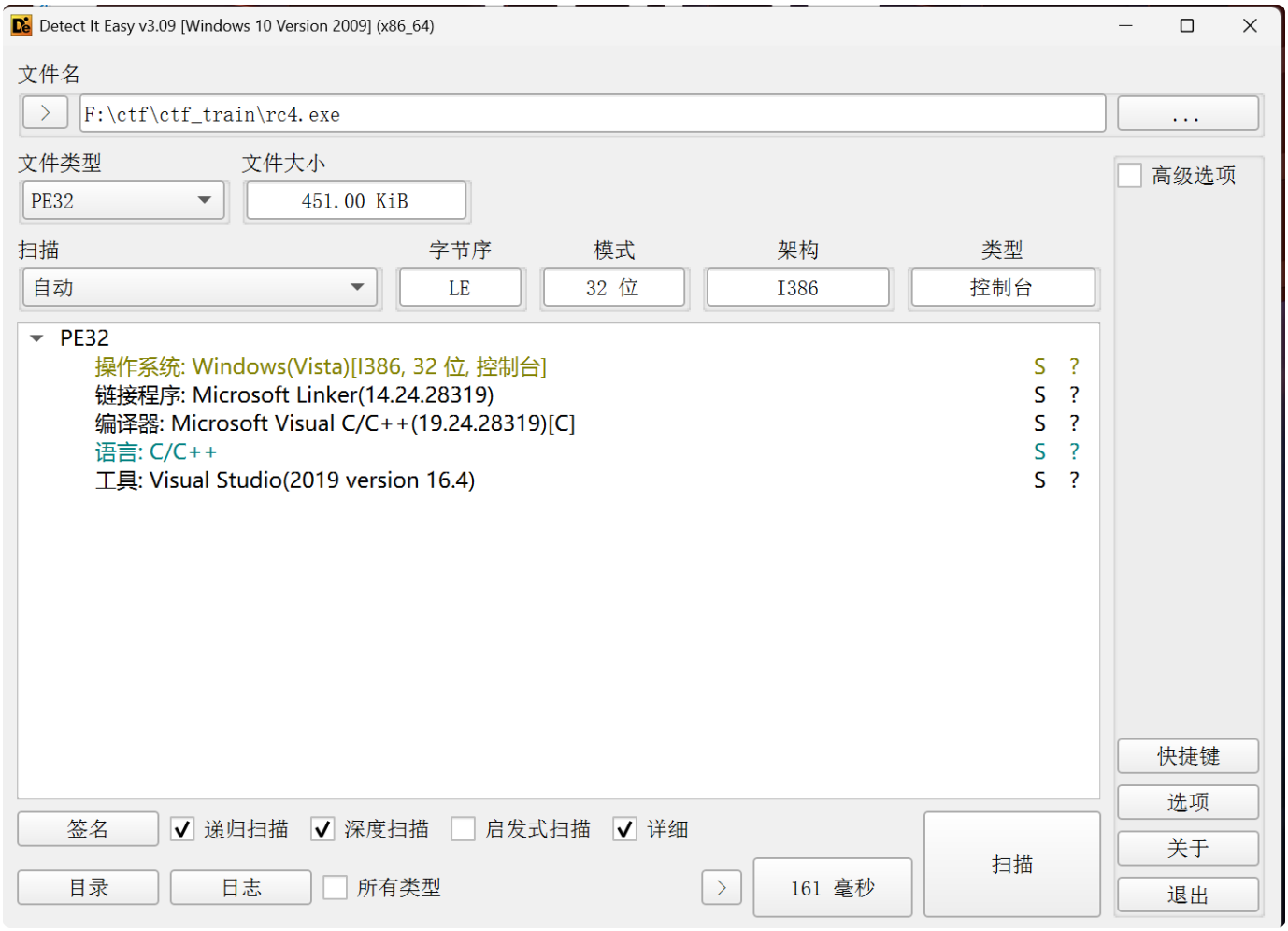
```
4   char input[264]; // [esp+3A4h] [ebp-12Ch] BYREF
5   char Str1[32]; // [esp+4ACh] [ebp-24h] BYREF
6
7   qmemcpy(Str1, &unk_50DE50, 28u);
8   printf("[6] Hi CTFer,Input your flag:");
9   scanf("%s", input);
10  for ( i = 0; i < j__strlen(Str1); ++i )
11    Str1[i] = ((i + 1) ^ Str1[i]) - i;
12  if ( !j__strcmp(Str1, input) )
13    printf("you are right\n");
14  else
15    printf("you are wrong\n");
16  return 0;
17 }
```

Str1在代码中不依赖任何输入即无论输入的是什么，最后输入的东西都会在这里与Str1进行比较，Str1是经过代码计算出来的，所以想要查看最后进行比较的是什么，就需要在代码运行到这一行时，查看内存，所以需要在这一行进行下断点，或者在前一行下断点，然后动态调试，F9运行到断点这一行代码F8

单步运行到这一行，然后查看解密后在内存中存了什么　　/*IDA右侧是寄存器窗口，底部是内存窗口，底部右侧是栈窗口*/

IDA动态调试秒RC4

DIE查看程序结构



PE32程序，无壳，C语言编写，用ida打开

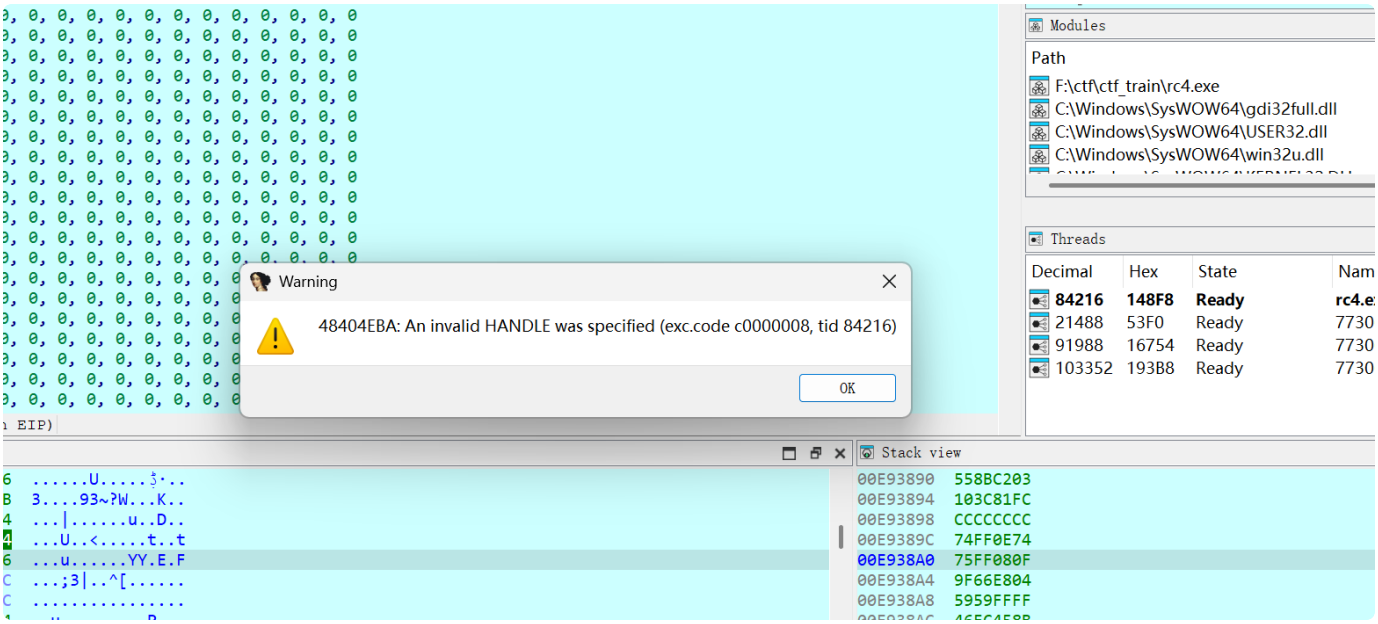
找到main函数并进入

```
  v7[38] = 36;
  v7[39] = -63;
  v7[40] = 24;
  v7[41] = -109;
  v7[42] = -77;
  v7[43] = -22;
  sub_42F057(v14);
  sub_42C40B("Welcome!! give me your flag:\n");
  do
  {
    char = j_j_j___fgetchar();
    if ( char == 10 )
      break;
    Str[n44++] = char;
  }
  while ( n44 < 44 );
  if ( (unsigned int)n44 >= 0x200 )
    j____report_rangecheckfailure();
  Str[n44] = 0;
  v6 = j__strlen(Str);
  sub_42CEFB(v13, v14, v6);
  for ( n256 = 0; n256 < 256; ++n256 )
    v12[n256] = v13[n256];
  sub_42D5B8(v13, Str, v6);
  for ( n44_1 = 0; n44_1 < 44; ++n44_1 )
  {
    if ( (char)v7[n44_1] == Str[n44_1] )
      ++n44_2;
  }
  if ( n44_2 == 44 )
    sub_42C40B("Yes, u right!\n");
  else
    sub_42C40B("no no no\n");
  sub_42D0CC("pause");
  return 0;
}
000073F9 _main_0:78 (432FF9)
```

将一些未知函数改为熟悉的

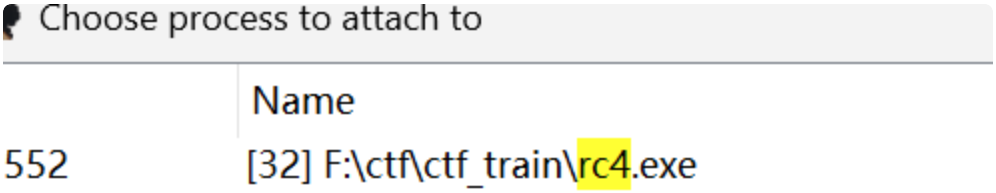
```
64  v7[42] = -77;
65  v7[43] = -22;
66  sub_42F057(v14);
67  printf("Welcome!! give me your flag:\n");
68  do
69  {
70    char = getchar();
71    if ( char == 10 )
72      break;
73    Str[n44++] = char;
74  }
75  while ( n44 < 44 );
76  if ( (unsigned int)n44 >= 0x200 )
77    j____report_rangecheckfailure();
78  Str[n44] = 0;
79  v6 = j__strlen(Str);
80  func1(v13, v14, v6);
81  for ( n256 = 0; n256 < 256; ++n256 )
82    v12[n256] = v13[n256];
83  sub_42D5B8(v13, Str, v6);
84  for ( n44_1 = 0; n44_1 < 44; ++n44_1 )
85  {
86    if ( (char)v7[n44_1] == Str[n44_1] )
87      ++n44_2;
88  }
89  if ( n44_2 == 44 )
90    printf("Yes, u right!\n");
91  else
92    printf("no no no\n");
93  sub_42D0CC("pause");
94  return 0;
95 }
```

尝试进行动态调试



有一个反调试，视频中讲，需要绕过这个反调试，先在计算机上运行程序





找到正在运行的程序，然后将这个正在运行的程序附属到ida中/*我的理解，就是这个反调试程序在（printf("Welcome!! give me your flag:\n");）之前，而当程序运行到这里时，再将它附属到ida中则ida会直接在这一行进行，则不会卡在那个反调试处，看到输出的东西说明反调试已经过了*/

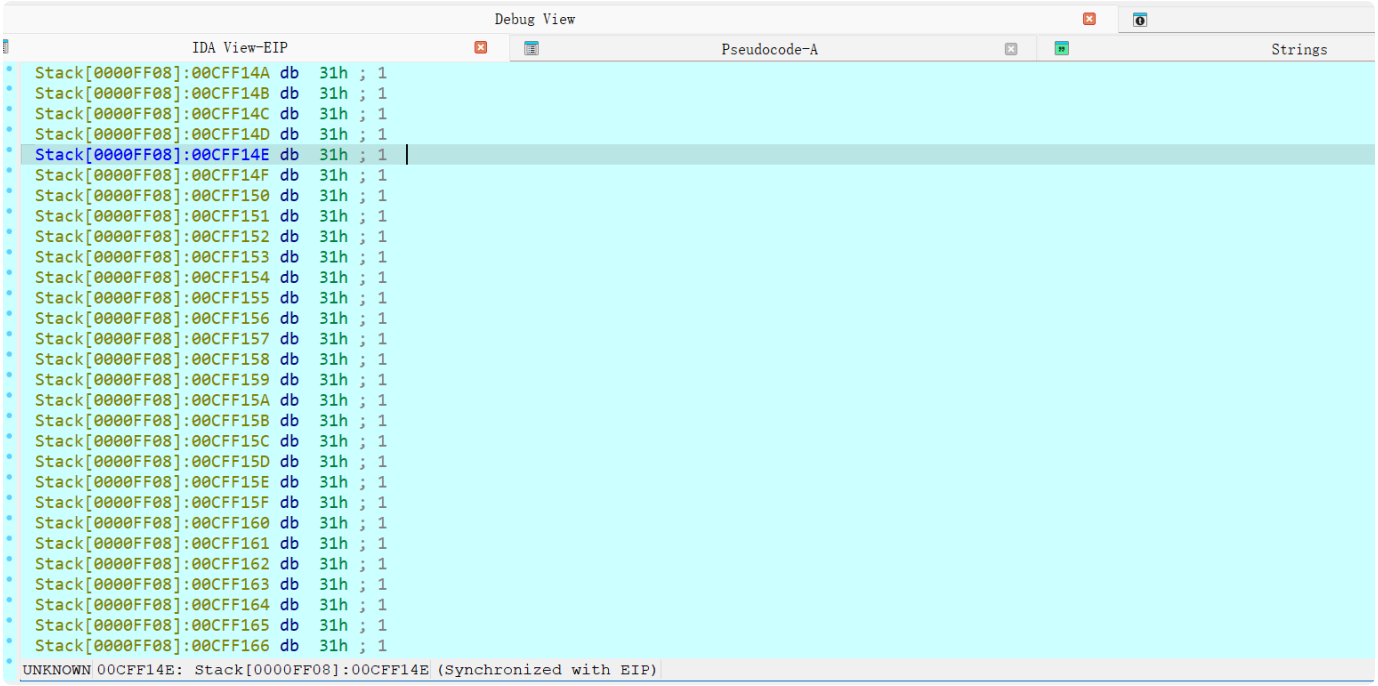
```
78  Str[n44] = 0;
79  v6 = j__strlen(Str);
80  func1(v13, v14, v6);
81  for ( n256 = 0; n256 < 256; ++n256 )
82    v12[n256] = v13[n256];
83  sub_E8D5B8(v13, Str, v6);
84  for ( n44_1 = 0; n44_1 < 44; ++n44_1 )
85  {
86    if ( (char)v7[n44_1] == Str[n44_1] )
87      ++n44_2;
88  }
89  if ( n44_2 == 44 )
```

先分析伪代码（if ((char)v7[n44_1] == Str[n44_1] ）这是一个if语句，就是当这两个数据相等时，则会进行输出you are right！ 所以它的上面第84行就是他的加密，我们在前一行第83行下断点然后动调

输入：11111111111111111111111111111111111111111111（44个1）

```
func1(v13, v14, v6);
for ( n256 = 0; n256 < 256; ++n256 )
  v12[n256] = v13[n256];
sub_80D5B8(v13, Str, v6);
for ( n44_1 = 0; n44_1 < 44; ++n44_1 )
{
  if ( (char)v7[n44_1] == Str[n44_1] )
    ++n44_2;
}
if ( n44_2 == 44 )
  printf("Yes, u right!\n");
else
  printf("no no no\n");
sub_80D0CC("pause");
return 0;
}
```

断点命中，



查看一下输入的数据，F8单步运行，（00CFF148）

我的ida没有加载插件正在解决成功了，日志已经出现：解决了

继续

经过加密后11111111111111111111111111111111111111111111111111（44个1）->7DB6A7C7741F7EF97D8DF3295498D5E9B4B8895B620FC958902BD01E431A94C78A253280FD61950893EBA600

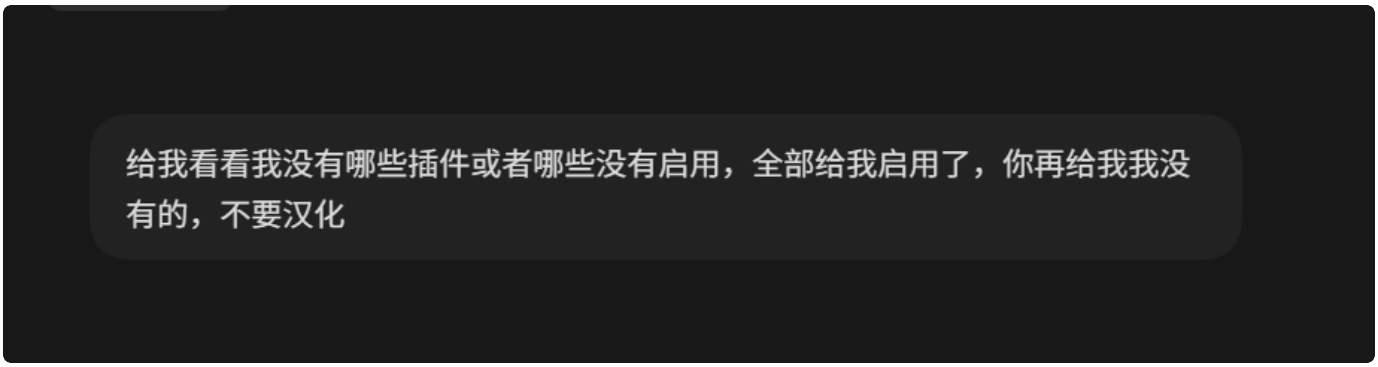
v7=E415C4EDA62F5610BB13EBAD7556C7BBBBE9B9CC023A509F369069BE7C4244CAC6D4245CD2B924C11893B3EACC给他经过加密会变成明文

再次运行程序动调，

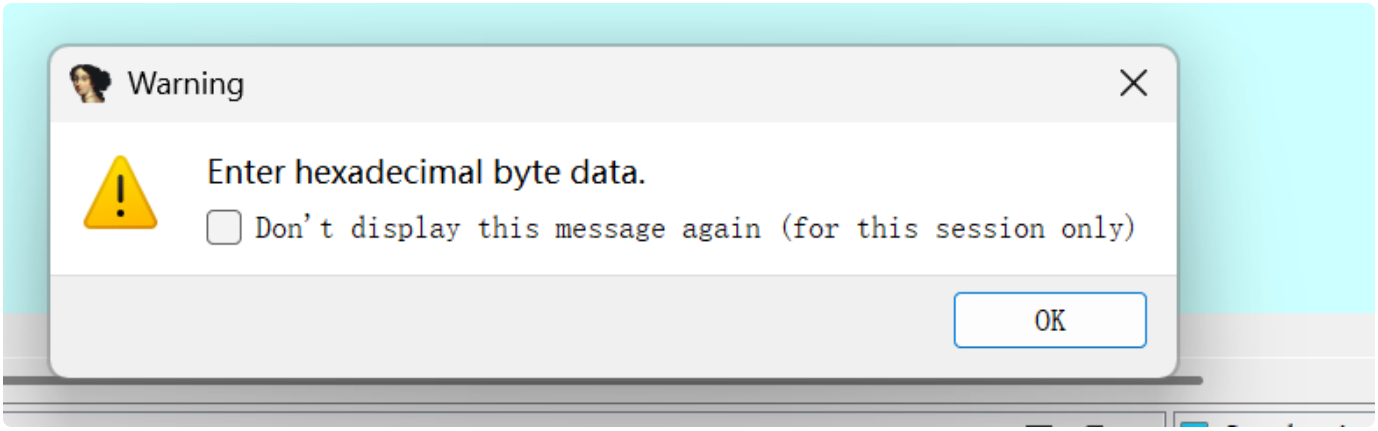
```
F6C3 db 0CCh
F6C4 db  31h ; 1
F6C5 db  31h ; 1
F6C6 db  31h ; 1
F6C7 db  31h ; 1
F6C8 db  31h ; 1
F6C9 db  31h ; 1
F6CA db  31h ; 1
F6CB db  31h ; 1
F6CC db  31h ; 1
F6CD db  31h ; 1
F6CE db  31h ; 1
F6CF db  31h ; 1
F6D0 db  31h ; 1
F6D1 db  31h ; 1
F6D2 db  31h ; 1
F6D3 db  31h ; 1
F6D4 db  31h ; 1
F6D5 db  31h ; 1
F6D6 db  31h ; 1
F6D7 db  31h ; 1
F6D8 db  31h ; 1
F6D9 db  31h ; 1
F6DA db  31h ; 1
F6DB db  31h ; 1
F6DC db  31h ; 1
F6DD db  31h ; 1
F6DE db  31h ; 1
F6DF db  31h ; 1
```

将这些改为v7

。。。又一个插件没启用，做不成了，让ai给我搞一下



已疯

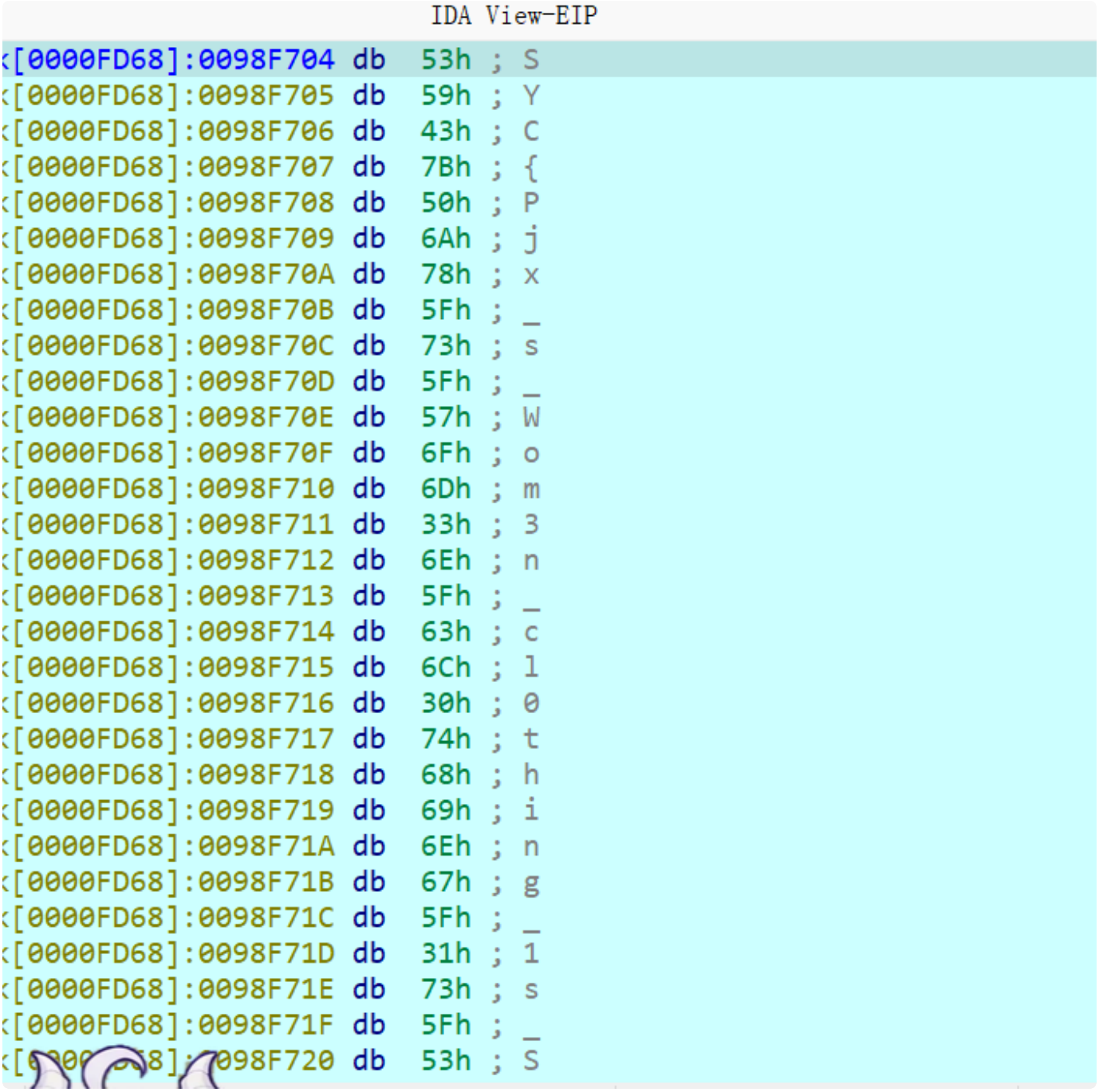


直接用python改

data=bytes.fromhex("E415C4EDA62F5610BB13EBAD7556C7BBBBE9B9CC023A509F369069BE7C4244CAC6D4245CD2B924C11893B3EACC"); n=ida_dbg.write_dbg_memory(0x0098F704,data); ida_dbg.invalidate_dbgmem_contents(0x0098F72F,len(data));

修改完之后F8单步，查看Str

```
79  v6 = j__strlen(Str);
80  sub_80CEFB(v13, v14, v6);
81  for ( n256 = 0; n256 < 256; ++n256 )
82    v12[n256] = v13[n256];
83  sub_80D5B8(v13, Str, v6);
84  for ( n44_1 = 0; n44_1 < 44; ++n44_1 )
85  {
86    if ( (char)v7[n44_1] == Str[n44_1] )
87      ++n44_2;
88  }
```



flag出来了/*已被IDA急哭*/

2026/10/22（10/20修复IDA插件问题，下坏了，重新下的byte老师的 10/21写作业，太多了，作文，英语测试）

IDA代码修复与数组识别(看不太懂)

IDA按F5反汇编出来的伪代码是IDAS自己猜出来的，并不准确，所以它猜错的地方需要人为进行修改

代码修复：常见问题与对应的处理办法

症状	*处理办法*	*操作*
变量类型不对	*改变量类型*	*光标放变量上按Y*
*结构体被拆成 *（a1+a4）.* (a1+8)*	*定义 struct，把参数类型改成结构体指针*	*Local Types / Structures 窗口 → Y*
参数个数/类型不对	*改函数原型*	*光标放函数名上按Y，写 int_fastcall check(chaar *s)*
相邻的 var__XX其实是数组	*定义数组*	*按 *填长度；或 Y 直接写 intt v[10]*
数据被当成代码	*取消定义后重新定义*	*先U（取消），再C（代码）或D（数据）或A（字符串）*
函数边界错、多出一段尾块	*重划边界*	*Edit > Functions > Set function end / Remove function tail*
call analysis failed	*调用约定错*	*右键 call 指令 → Force call type，或改函数原型*
sp-analysis failed	*栈失衡*	*Alt+K打开栈帧手动改；**初学者直接以汇编为准，别花时间强行修复***

我听得并不太懂，虽然知道如何做但我不知道如何分析

必须记熟的快捷键

键	**作用**
F5 / Tab	**反编译 / 汇编<-->伪代码切换**
N	**重命名（最高频）**
Y	**设置类型（第二高频）**
*	**定义数组**

X	交叉引用（谁调用/引用了它）—— 逆向最重要的键
G / Esc	跳转地址 / 回退
C / D / A / U	代码 / 数据 / 字符串 / 取消定义
P / Alt+P	创建函数 / 编辑函数
Ctrl+K	栈帧窗口（看局部变量布局）
;	加注释
Shift+F12	字符串窗口（找突破口的第一步）

2026/10/23

[NISACTF 2022]string

比较简单，是只要长度正确，就会输出flag

DIE打开时ELF文件，采用IDA动态调试

```
C:\Users\tom>wsl
tom@LAPTOP-H8GEUFSD:/mnt/c/Users/tom$ ./linux_server
```

IDA



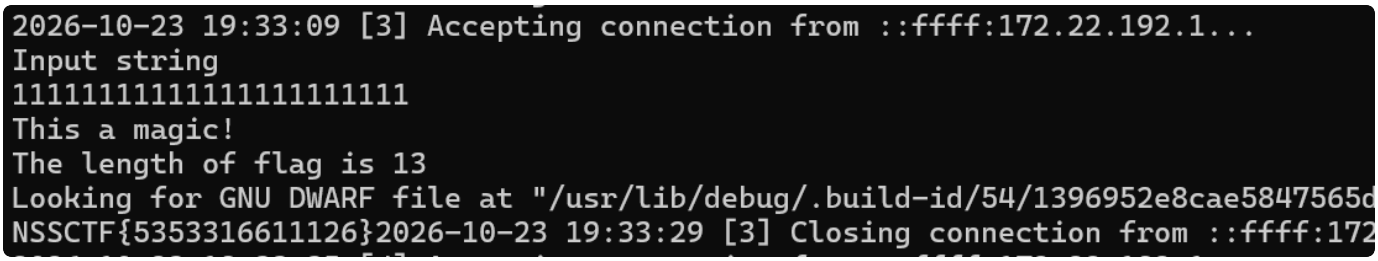
```
26        goto LABEL_4;
27      }
28    }
29    else
30    {
31  LABEL_4:
32      for ( i = (int)v6; ((i - 16843009) & ~i & 0x80808080) == 0; i = v13 )
33      {
34        v13 = v6[1];
35        v6 += 4;
36      }
37      v2 = v6++;
38      for ( j = *v2; j; j = *v3 )
39        v3 = v6++;
40    }
41    puts("This a magic!");
42    n13 = (_DWORD)v6 - v12;
43    for ( k = 0; (int)v6 - v12 > k; ++k )
44      n13 ^= 0x1Au;
45    if ( n13 != 13 )
46    {
47      puts("error!");
48      exit(0);
49    }
50    puts("The length of flag is 13");
51    srand(seed);
52    printf("NSSCTF{");
53    for ( n13_1 = 0; n13_1 < 13; ++n13_1 )
54    {
55      v4 = rand();
56      printf("%d", v4 % 8 + 1);
57    }
58    putchar(125);
59    return &v6[-v12];
60  }
```

写一个简单的脚本

```c
#include <stdio.h>
#include <string.h>

int main() {
    int n13 = 13;
    n13 ^= 0x1Au;
    printf("Result: %d\n", n13);
    return 0;
}
```

就OK了。输入23位



2026/10/24~25

脖子落枕了这两天学的比较少

主要还是进行极客大挑战

我只做了Re的题

wp还没写